

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA TOÁN - CƠ - TIN HỌC



BÁO CÁO CUỐI KÌ

Tên đề tài: **PHÂN LOẠI MỨC ĐỘ BÉO PHÌ DỰA TRÊN
THÓI QUEN SINH HOẠT HÀNG NGÀY**

Nhóm thực hiện: Nhóm 12

MACHINE LEARNING

Hà Nội - 05/2026

BÁO CÁO CUỐI KÌ

Học máy

Giảng viên hướng dẫn: TS. Cao Văn Chung

Sinh viên thực hiện: Nguyễn Vũ Dương - 23001964

Bùi Tiến Bình - 23000095

Nguyễn Mạnh Giang - 23000113

Hà Nội – Tháng 5/2026

Lời cảm ơn

Để hoàn thành tốt đẹp đề tài này, bên cạnh sự nỗ lực không ngừng của các thành viên trong nhóm, chúng em đã vô cùng may mắn nhận được sự quan tâm, động viên và giúp đỡ quý báu từ nhiều tập thể và các cá nhân. Trước hết, chúng em xin gửi tới toàn thể các thầy, cô giáo dạy môn Machine Learning. Đặc biệt, chúng em cũng xin bày tỏ lòng biết ơn sâu sắc nhất tới Giảng viên TS. Cao Văn Chung, người thầy đã tận tình hướng dẫn, chỉ bảo chúng em trong suốt quá trình nghiên cứu và hoàn thiện đề tài. Do kiến thức và kinh nghiệm thực tiễn còn hạn chế, đề tài của nhóm chúng em không tránh khỏi những thiếu sót. Kính mong nhận được những ý kiến đóng góp quý báu từ quý thầy cô để đề tài được hoàn thiện hơn nữa. Cuối cùng, chúng em xin kính chúc quý thầy cô luôn dồi dào sức khỏe, hạnh phúc và gặt hái thêm nhiều thành công rực rỡ trong sự nghiệp “trồng người” cao cả của mình.

Tóm tắt nội dung

Đề tài này tập trung nghiên cứu và xây dựng các mô hình học máy nhằm phân loại mức độ béo phì của từng cá nhân dựa trên các đặc trưng liên quan đến thói quen sinh hoạt hằng ngày và các chỉ số thể chất. Cụ thể, nhóm tiến hành áp dụng một số thuật toán học máy tiêu biểu gồm K-Nearest Neighbors (KNN), Softmax Regression và Support Vector Machine (SVM). Trong đó, Softmax Regression được lựa chọn nhờ khả năng mô hình hóa xác suất, tính đơn giản và dễ diễn giải, phù hợp với các bài toán phân loại đa lớp có mối quan hệ tuyến tính. Trong khi đó, SVM nổi bật với khả năng xác định siêu phẳng phân tách tối ưu và hoạt động hiệu quả trong không gian đặc trưng có số chiều cao, đặc biệt khi tồn tại sự chồng lấn giữa các lớp dữ liệu. KNN được sử dụng như một phương pháp phân loại dựa trên khoảng cách, cho phép đánh giá nhãn của một đối tượng dựa trên các điểm dữ liệu lân cận gần nhất.

Quy trình thực hiện bao gồm các bước chính như tiền xử lý dữ liệu, mã hóa các thuộc tính định tính, chuẩn hóa dữ liệu và chia tập dữ liệu thành các tập huấn luyện, kiểm tra và đánh giá. Sau đó, các mô hình được huấn luyện trên tập dữ liệu đã xử lý và đánh giá hiệu quả thông qua các chỉ số như Accuracy, F1-score và ma trận nhầm lẫn.

Kết quả thực nghiệm cho thấy các mô hình đạt độ chính xác cao nhất là hơn 90%, qua đó chứng minh hiệu quả của các phương pháp học máy trong bài toán phân loại mức độ béo phì. Báo cáo nhấn mạnh tiềm năng ứng dụng của các mô hình này trong lĩnh vực hỗ trợ chẩn đoán y khoa, góp phần hỗ trợ bác sĩ đưa ra quyết định nhanh chóng và chính xác hơn. Đồng thời, nghiên cứu cũng mở ra các hướng phát triển tiếp theo như tối ưu siêu tham số, lựa chọn đặc trưng và kết hợp nhiều mô hình nhằm nâng cao hiệu suất phân loại.

Mục lục

1	Tiền xử lý dữ và giảm chiều dữ liệu	4
1.1	Tổng quan bộ dữ liệu	4
1.2	Mã hóa dữ liệu	5
1.2.1	Mã hóa đặc trưng bằng Binary Encoding	5
1.2.2	Mã hóa đặc trưng bằng Ordinal Encoding	6
1.3	Chuẩn hóa dữ liệu	6
1.3.1	Chuẩn hóa bằng StandardScaler	6
1.3.2	Chuẩn hóa bằng RobustScaler	7
1.4	Phân tích thành phần chính (PCA)	8
1.4.1	Cơ sở lý thuyết	8
1.4.2	Trực quan hóa dữ liệu sau PCA và đánh giá kết quả	10
1.5	Phân tích phân biệt tuyến tính (LDA)	13
1.5.1	Cơ sở lý thuyết	14
1.5.2	Trực quan hóa dữ liệu và đánh giá kết quả	17
1.6	So sánh PCA với LDA	18
2	Các mô hình phân loại, phân cụm và chuyển về dạng hồi quy	20
2.1	Thuật toán phân cụm K-means	20
2.1.1	Hàm mất mát và bài toán tối ưu	20
2.1.2	Thuật toán tối ưu hàm mất mát	21
2.1.3	Trực quan hóa và đánh giá kết quả	23
2.2	K-Nearest neighbors (KNN)	27
2.2.1	Giới thiệu	27
2.2.2	Cách thuật toán hoạt động	27
2.2.3	Bài toán phân loại đa lớp	28
2.2.4	Hàm tính khoảng cách	28

2.2.5	Tìm K láng giềng gần nhất và dự đoán nhãn	29
2.2.6	KNN có trọng số	30
2.2.7	Lựa chọn tham số K	31
2.3	Softmax Regression	31
2.3.1	Mô hình hóa xác suất	31
2.3.2	Hàm Softmax	32
2.3.3	Quyết định lớp dựa trên xác suất	32
2.3.4	Hàm mất mát (Loss Function)	33
2.3.5	Tối ưu hóa mô hình	33
2.4	Support vector machine (SVM)	34
2.4.1	Giới thiệu	34
2.4.2	Một số khái niệm cơ bản	34
2.4.3	Bài toán tối ưu của SVM	35
2.5	Chuyển đổi từ SVM sang SVM hồi quy (SVR)	38
2.5.1	Support Vector Regression (SVR)	38
2.5.2	Chuyển đổi từ SVM phân loại sang SVR hồi quy	39
2.5.3	Cách áp dụng mô hình SVR từ mô hình SVM phân loại	39
2.5.4	Các khái niệm cốt lõi trong SVR	40
2.5.5	Ý nghĩa của phương pháp	40
2.6	Chuyển đổi từ Softmax phân loại sang Softmax hồi quy	41
2.6.1	Giới thiệu phương pháp Softmax hồi quy	41
2.6.2	Chuyển đổi từ Softmax Classification sang hồi quy	41
2.6.3	Cách áp dụng hồi quy từ mô hình Softmax phân loại	42
2.6.4	Ý nghĩa của phương pháp	42
3	Nhận xét và đánh giá kết quả thực nghiệm	43
3.1	Đánh giá mô hình phân loại	43
3.2	Đánh giá mô hình hồi quy	49
3.3	Một số hạn chế và phương hướng phát triển	52

Lý do chọn đề tài

Trong bối cảnh xã hội hiện đại, tình trạng béo phì đang gia tăng nhanh chóng và trở thành một trong những vấn đề sức khỏe cộng đồng đáng lo ngại trên toàn cầu. Béo phì không chỉ ảnh hưởng tiêu cực đến chất lượng cuộc sống mà còn là nguyên nhân dẫn đến nhiều bệnh lý nguy hiểm như tim mạch, tiểu đường và rối loạn chuyển hóa, gây áp lực lớn lên hệ thống y tế và kinh tế xã hội. Việc đánh giá và phân loại mức độ béo phì một cách chính xác, kịp thời đóng vai trò then chốt trong công tác phòng ngừa, theo dõi và điều trị hiệu quả cho người bệnh.

Tuy nhiên, các phương pháp đánh giá truyền thống hiện nay thường mang tính thủ công, phụ thuộc nhiều vào kinh nghiệm của chuyên gia và khó triển khai trên quy mô lớn, đặc biệt trong bối cảnh dữ liệu y tế ngày càng gia tăng cả về số lượng lẫn độ phức tạp. Bên cạnh đó, những phương pháp này đôi khi chưa khai thác hết mối quan hệ tiềm ẩn giữa các yếu tố như thói quen sinh hoạt, chế độ ăn uống và các chỉ số thể chất.

Trong bối cảnh đó, việc ứng dụng các phương pháp học máy để tự động hóa quá trình phân loại mức độ béo phì đang nổi lên như một hướng tiếp cận hiệu quả và đầy tiềm năng. Các mô hình học máy không chỉ có khả năng xử lý khối lượng dữ liệu lớn mà còn phát hiện các mẫu và xu hướng ẩn mà con người khó nhận biết. Điều này góp phần nâng cao độ chính xác trong chẩn đoán, hỗ trợ ra quyết định nhanh chóng và mở rộng khả năng ứng dụng trong thực tiễn, đặc biệt trong các hệ thống hỗ trợ y tế thông minh.

Chương 1

Tiền xử lý dữ và giảm chiều dữ liệu

1.1 Tổng quan bộ dữ liệu

Bộ dữ liệu bao gồm 17 thuộc tính, mô tả các đặc điểm cá nhân và thói quen sinh hoạt của bệnh nhân, cụ thể như sau:

1. **Gender**: Giới tính của bệnh nhân.
2. **Age**: Tuổi của bệnh nhân.
3. **Height**: Chiều cao của bệnh nhân.
4. **Weight**: Cân nặng của bệnh nhân.
5. **family_history_with_overweight**: Tiền sử béo phì trong gia đình.
6. **FAVC** (Frequent Consumption of High Caloric Food): Tần suất tiêu thụ thực phẩm giàu năng lượng.
7. **FCVC** (Frequency of Consumption of Vegetables): Tần suất tiêu thụ rau, củ, quả.
8. **NCP** (Number of Main Meals): Số bữa ăn chính mỗi ngày.
9. **CAEC** (Consumption of Food Between Meals): Thói quen ăn thêm giữa các bữa chính.
10. **SMOKE**: Tình trạng hút thuốc của bệnh nhân.
11. **CH2O** (Consumption of Water): Lượng nước tiêu thụ hàng ngày.
12. **SCC** (Calories Monitoring): Việc theo dõi lượng calo tiêu thụ.

13. **FAF** (Physical Activity Frequency): Tần suất hoạt động thể chất.
14. **TUE** (Time Using Technology Devices): Thời gian sử dụng thiết bị công nghệ.
15. **CALC** (Alcohol Consumption): Tần suất tiêu thụ đồ uống có cồn.
16. **MTRANS** (Mode of Transportation): Phương tiện di chuyển hằng ngày.
17. **NObeyesdad**: biến mục tiêu gồm 7 loại béo phì
 - *Insufficient Weight*
 - *Normal Weight*
 - *Overweight Level I*
 - *Overweight Level II*
 - *Obesity Type I*
 - *Obesity Type II*
 - *Obesity Type III*

Những thuộc tính này đóng vai trò quan trọng trong việc phân tích và chẩn đoán mức độ béo phì của bệnh nhân. Đặc biệt, chúng cho phép khai thác mối quan hệ giữa các chỉ số cơ thể và thói quen sinh hoạt, từ đó hỗ trợ phân biệt giữa các mức độ béo phì khác nhau một cách hiệu quả và có hệ thống.

1.2 Mã hóa dữ liệu

1.2.1 Mã hóa đặc trưng bằng Binary Encoding

Đối với các đặc trưng dạng nhị phân (binary categorical features), việc chuyển đổi sang dạng số là cần thiết để các mô hình học máy có thể xử lý hiệu quả. Trong nghiên cứu này, các đặc trưng bao gồm **FAVC**, **SCC**, **SMOKE**, **family_history_with_overweight**, **Gender** được mã hóa bằng phương pháp *Binary Encoding*.

Cụ thể, các giá trị dạng chuỗi được ánh xạ về dạng số như sau: các giá trị mang ý nghĩa khẳng định như *Yes*, *yes*, *Male* được gán giá trị 1, trong khi các giá trị mang ý nghĩa phủ định như *No*, *no*, *Female* được gán giá trị 0.

Việc mã hóa này giúp đơn giản hóa dữ liệu, đồng thời giữ nguyên ý nghĩa logic của các biến nhị phân. Nhờ đó, các thuật toán học máy có thể khai thác hiệu quả mối quan

hệ giữa các đặc trưng đầu vào và biến mục tiêu, góp phần nâng cao độ chính xác của mô hình.

1.2.2 Mã hóa đặc trưng bằng Ordinal Encoding

Đối với các đặc trưng có tính chất thứ bậc (ordinal), việc sử dụng *Ordinal Encoding* là phù hợp nhằm chuyển đổi các giá trị dạng phân loại sang dạng số mà vẫn giữ được mối quan hệ thứ tự giữa các mức giá trị.

Trong nghiên cứu này, các đặc trưng **MTRANS**, **CALC**, **CAEC** và biến mục tiêu **NObeyesdad** được mã hóa theo phương pháp này. Các biến này đều mang ý nghĩa tăng dần về mức độ hoặc tần suất, ví dụ như mức độ tiêu thụ (từ thấp đến cao) hoặc mức độ béo phì (từ nhẹ đến nặng).

Cụ thể:

- **CAEC, CALC:** được mã hóa theo mức độ tiêu thụ giảm dần (ví dụ: *Always* > *Frequently* > *Sometimes* > *No*).
- **MTRANS:** được sắp xếp theo mức độ vận động giảm dần (ví dụ: *Bike* > *Walking* > *Public_Transportation* > *Motorbike* > *Automobile*), phản ánh mức độ hoạt động thể chất.
- **NObeyesdad:** được mã hóa theo thứ tự từ 0 -> 6 theo mức độ béo phì (từ *Insufficient_Weight* đến *Obesity_Type_III*).

Việc sử dụng Ordinal Encoding giúp mô hình học máy khai thác được thông tin về thứ tự giữa các giá trị, từ đó cải thiện khả năng học và dự đoán so với việc mã hóa không xét đến thứ bậc.

1.3 Chuẩn hóa dữ liệu

1.3.1 Chuẩn hóa bằng StandardScaler

việc chuẩn hóa các đặc trưng là cần thiết nhằm đưa các biến về cùng một thang đo, từ đó giúp các mô hình học máy hoạt động hiệu quả hơn. Trong nghiên cứu này, nhóm sử dụng Phương pháp chuẩn hóa *Standard* cho các đặc trưng **Height**, **Weight**, **CH2O**, **FAF**, **FCVC**, **TUE**.

Phương pháp chuẩn hóa *Standard* chuẩn hóa dữ liệu theo phân phối chuẩn với trung bình bằng 0 và độ lệch chuẩn bằng 1. Cụ thể, mỗi giá trị x được biến đổi thành x_{scaled} theo công thức:

$$x_{\text{scaled}} = \frac{x - \mu}{\sigma}$$

Trong đó:

- x là giá trị ban đầu của dữ liệu.
- μ là giá trị trung bình của đặc trưng:

$$\mu = \frac{1}{N} \sum_{i=1}^N x_i$$

- σ là độ lệch chuẩn của đặc trưng:

$$\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2}$$

- N là số lượng mẫu dữ liệu.

Việc chuẩn hóa giúp giảm ảnh hưởng của sự khác biệt về đơn vị đo giữa các đặc trưng, đồng thời cải thiện tốc độ hội tụ và hiệu suất của các thuật toán như Logistic Regression và Support Vector Machine.

1.3.2 Chuẩn hóa bằng RobustScaler

Trong trường hợp dữ liệu chứa nhiều giá trị ngoại lai (outliers) hoặc có phân phối lệch (skewed distribution), việc sử dụng các phương pháp chuẩn hóa dựa trên trung bình và độ lệch chuẩn như StandardScaler có thể không hiệu quả. Do đó, trong nghiên cứu này, nhóm sử dụng phương pháp chuẩn hóa *Robust* cho các đặc trưng **Age** và **NCP**, do hai đặc trưng này có độ lệch cao và xuất hiện nhiều giá trị ngoại lai.

Phương pháp chuẩn hóa *Robust* thực hiện chuẩn hóa dữ liệu dựa trên trung vị (median) và khoảng tứ phân vị (Interquartile Range - IQR), giúp giảm ảnh hưởng của các giá trị ngoại lai. Cụ thể, mỗi giá trị x được biến đổi thành x_{scaled} theo công thức:

$$x_{\text{scaled}} = \frac{x - Q_2}{Q_3 - Q_1}$$

Trong đó:

- x là giá trị ban đầu của dữ liệu.
- Q_2 là trung vị (median), tương ứng với phân vị 50%.
- Q_1 là phân vị thứ nhất (25%).
- Q_3 là phân vị thứ ba (75%).
- $IQR = Q_3 - Q_1$ là khoảng tứ phân vị, phản ánh độ phân tán của dữ liệu.

Việc sử dụng RobustScaler giúp giữ được cấu trúc phân bố chính của dữ liệu mà không bị ảnh hưởng mạnh bởi các giá trị ngoại lai, từ đó cải thiện độ ổn định và hiệu quả của mô hình học máy.

Giảm chiều và trực quan hóa dữ liệu

1.4 Phân tích thành phần chính (PCA)

Phân tích thành phần chính (PCA) là phương pháp giảm chiều không giám sát, nhằm loại bỏ dư thừa và giữ lại các thành phần mang nhiều thông tin nhất. PCA biến đổi dữ liệu sang không gian mới với các trục tọa độ ít tương quan, giúp giảm chi phí tính toán, cải thiện hiệu suất mô hình và hỗ trợ trực quan hóa trong các bài toán dữ liệu có số chiều lớn.

1.4.1 Cơ sở lý thuyết

$$\begin{array}{c}
 \begin{array}{ccc}
 \begin{array}{|c|} \hline N \\ \hline D \\ \hline \mathbf{X} \\ \hline \end{array} & = & \begin{array}{|c|c|} \hline K & D-K \\ \hline D & \mathbf{U}_K & \bar{\mathbf{U}}_K \\ \hline \end{array} \times \begin{array}{|c|} \hline N \\ \hline K & \mathbf{Z} \\ \hline D-K & \mathbf{Y} \\ \hline \end{array} \\
 \text{Original data} & & \text{An orthogonal matrix} \quad \text{Coordinates in new basis} \\
 \\
 & = & \begin{array}{|c|} \hline K \\ \hline D & \mathbf{U}_K \\ \hline \end{array} \times \begin{array}{|c|} \hline N \\ \hline K & \mathbf{Z} & D \\ \hline \end{array} + \begin{array}{|c|} \hline \bar{\mathbf{U}}_K \\ \hline \end{array} \times \begin{array}{|c|} \hline \mathbf{Y} \\ \hline \end{array}
 \end{array}
 \end{array}$$

Hình 1.1: Minh họa nguyên lý giảm chiều của phương pháp PCA

Trong hình minh họa, hệ cơ sở mới $U = [U_k, \bar{U}_k]$ là hệ trực chuẩn với U_k là ma trận con tạo bởi k cột đầu tiên của U .

Mục đích của PCA là đi tìm ma trận trực giao U sao cho phần lớn thông tin được giữ lại ở phần $U_k Z$ (phần màu xanh) và có thể lược bỏ phần $\bar{U}_k Y$ (phần màu đỏ) và thay bằng một ma trận không phụ thuộc vào điểm dữ liệu.

Các bước thực hiện

1. Bước 1: Tính vector kỳ vọng của toàn bộ dữ liệu:

$$\bar{x} = \frac{1}{N} \sum_{n=1}^N x_n$$

Trong đó N là số lượng mẫu, và x_n là vector dữ liệu thứ n .

2. Bước 2: Tính dữ liệu chuẩn hóa $\hat{x}_n$:

$$\hat{x}_n = x_n - \bar{x}, \quad n = 1, 2, \dots, N.$$

3. Bước 3: Tính ma trận hiệp phương sai:

$$S = \frac{1}{N} \hat{X} \hat{X}^T$$

Trong đó $\hat{X}$ là ma trận dữ liệu đã được chuẩn hóa.

4. Bước 4: Tính các trị riêng λ_i và vector riêng u_i của ma trận hiệp phương sai.

5. Bước 5: Chọn k giá trị riêng lớn nhất và k vector riêng tương ứng để xây dựng ma trận U_k .

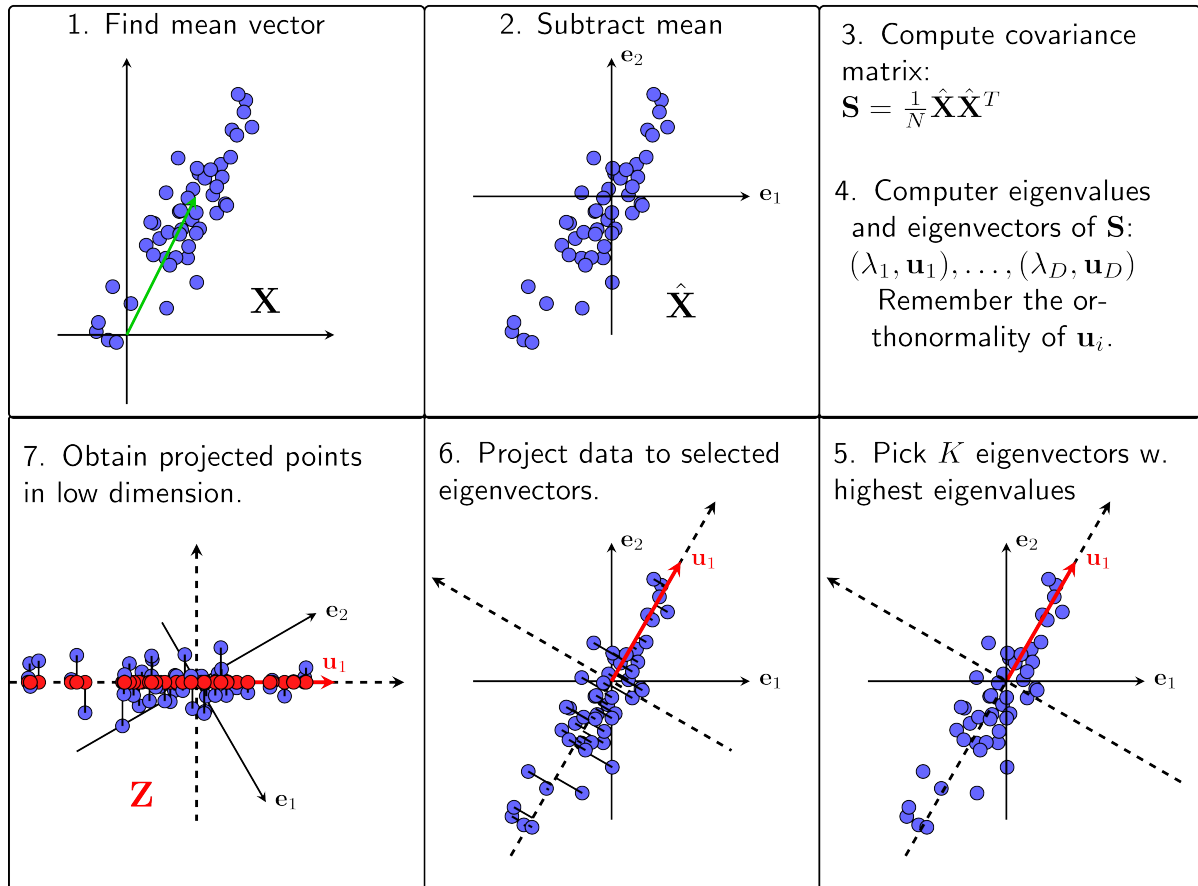
Tập hợp các vector này được dùng để tạo thành không gian con mới có số chiều nhỏ hơn.

6. Bước 6: Các cột của $\{U_k\}_{j=1}^k$ là hệ cơ sở trực chuẩn.

7. Bước 7: Chiếu dữ liệu ban đầu đã chuẩn hóa $\hat{X}$ xuống không gian con mới:

$$Z = U_k^T \hat{X}$$

PCA procedure



Hình 1.2: Minh họa các bước giảm chiều của phương pháp PCA

Ma trận Z chứa các biểu diễn nén của dữ liệu trên không gian có số chiều thấp hơn.

Dữ liệu ban đầu có thể được xấp xỉ lại từ dữ liệu nén như sau:

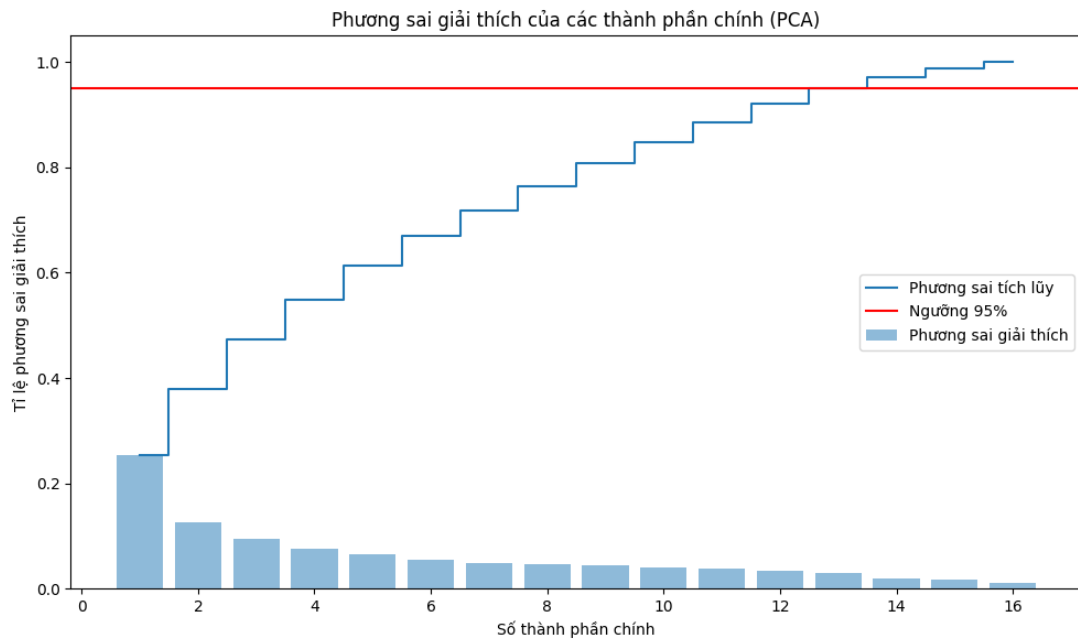
$$x \approx U_k Z + \bar{x}$$

Trong đó $U_k Z$ là phần thông tin được giữ lại, và $\bar{x}$ là trung bình của dữ liệu ban đầu, được cộng lại để khôi phục vị trí gốc của dữ liệu.

1.4.2 Trực quan hóa dữ liệu sau PCA và đánh giá kết quả

Để đảm bảo phương pháp PCA vừa giảm được số chiều dữ liệu vừa giữ lại phần lớn thông tin quan trọng, cần lựa chọn số lượng thành phần chính một cách hợp lý. Trong nghiên cứu này, số thành phần chính được xác định dựa trên tổng phương sai tích lũy (cumulative explained variance). Cụ thể, nhóm lựa chọn số thành phần chính nhỏ nhất sao cho tổng phương sai giữ lại đạt từ 95% trở lên. Tiêu chí này giúp đảm bảo rằng phần

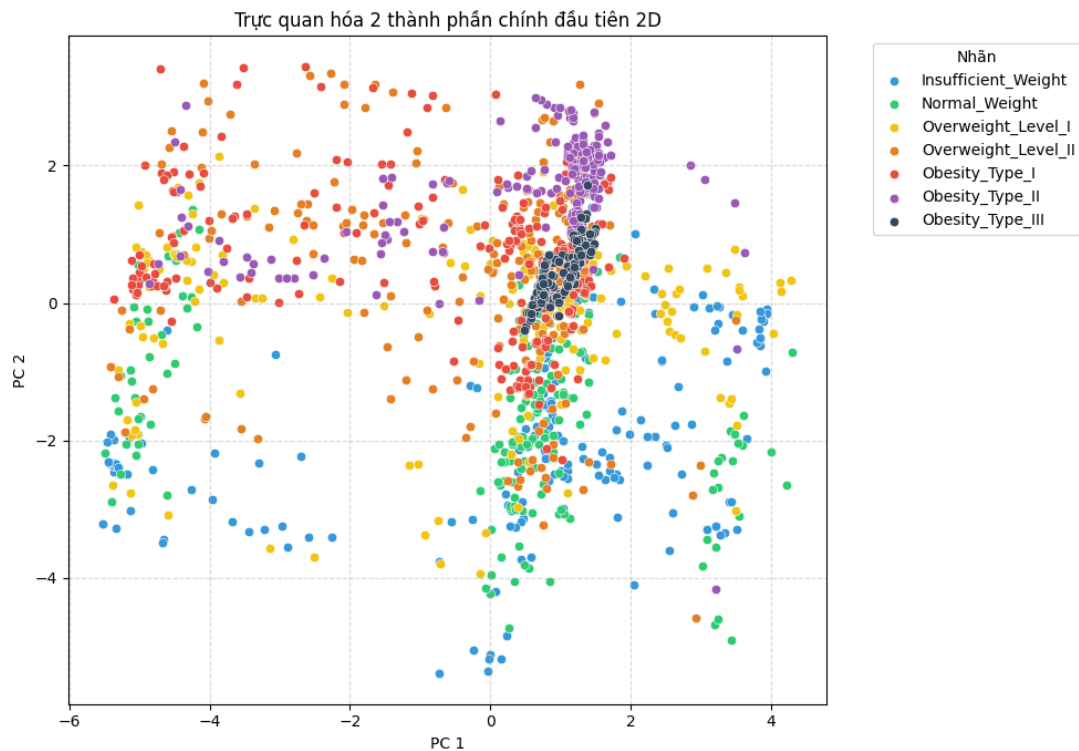
lớn thông tin của dữ liệu gốc được bảo toàn, đồng thời giảm thiểu số chiều không cần thiết, góp phần cải thiện hiệu quả tính toán và hiệu suất của mô hình.



Hình 1.3: Biểu đồ phương sai giải thích tích lũy

Có thể thấy rằng tổng phương sai tích lũy tăng nhanh trong những thành phần chính đầu tiên và dần ổn định khi số chiều tăng lên. Ngưỡng 95% phương sai được đạt tại khoảng 13 thành phần chính. Do đó, nhóm quyết định giữ lại 13 thành phần chính đầu tiên để đảm bảo cân bằng giữa việc giảm chiều dữ liệu và bảo toàn thông tin.

Sau khi áp dụng phương pháp PCA lên dữ liệu, ta thu được kết quả trực quan hóa như sau:



Hình 1.4: Biểu diễn dữ liệu trên hai thành phần chính đầu tiên

Biểu đồ PCA trên thể hiện sự phân bố của dữ liệu theo 7 loại bệnh béo phì, bao gồm:

Nhóm 0 – *Insufficient Weight* : Xanh dương

Nhóm 1 – *Normal Weight* : Cam

Nhóm 2 – *Overweight Level I* : Xanh lá cây

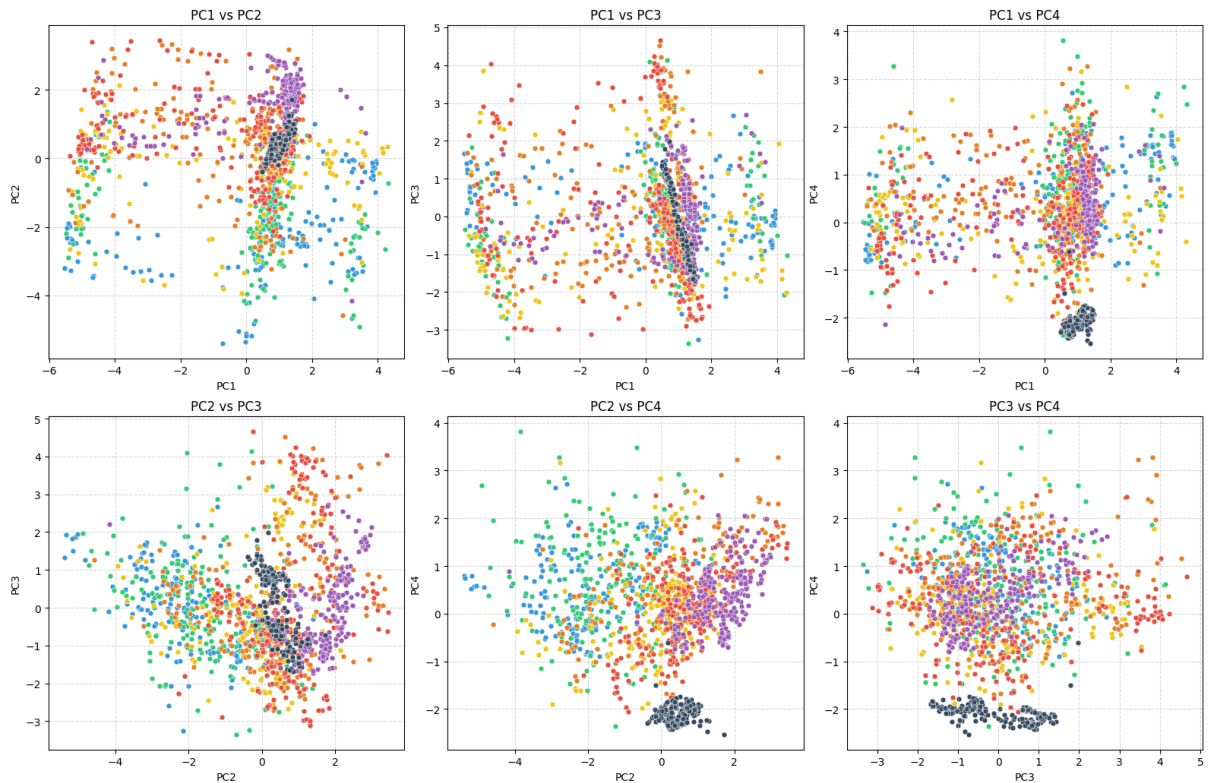
Nhóm 3 – *Overweight Level II* : Đỏ

Nhóm 4 – *Obesity Type I* : Hồng

Nhóm 5 – *Obesity Type II* : Nâu

Nhóm 6 – *Obesity Type III* : Xám

Trực quan hóa dữ liệu trên các thành phần chính từ PC1 đến PC4 được thể hiện trong Hình ??.



Hình 1.5: Biểu đồ phân tán của bốn thành phần chính đầu tiên

Nhận xét: Qua kết quả sau khi giảm chiều bằng PCA, có thể thấy rằng ngay cả khi xét đến bốn thành phần chính đầu tiên, khả năng phân tách giữa các nhóm dữ liệu vẫn chưa rõ ràng. Các điểm dữ liệu thuộc các lớp khác nhau tiếp tục có sự chồng lấn đáng kể trên không gian biểu diễn.

Điều này cho thấy rằng các thành phần chính được trích xuất chủ yếu nhằm tối đa phương sai của dữ liệu, nhưng không đảm bảo tối ưu hóa khả năng phân biệt giữa các lớp. Do đó, PCA chưa thực sự hiệu quả đối với bài toán phân loại trong trường hợp này.

Do đó, đối với bộ dữ liệu này, PCA chưa phải là phương pháp giảm chiều hiệu quả nếu mục tiêu chính là phục vụ cho bài toán phân loại.

1.5 Phân tích phân biệt tuyến tính (LDA)

Phân tích phân biệt tuyến tính (LDA) là một phương pháp giảm chiều dữ liệu có giám sát, nhằm tối đa hóa khả năng phân biệt giữa các nhóm dữ liệu đã được gán nhãn. Khác với PCA, LDA sử dụng thông tin về nhãn lớp để xác định các hướng chiếu tối ưu trong không gian đặc trưng.

Mục tiêu của LDA là tìm các trục chiếu sao cho khoảng cách giữa các lớp được tối đa hóa, đồng thời độ phân tán bên trong mỗi lớp được tối thiểu hóa. Nhờ đó, dữ liệu sau

khi giảm chiều có xu hướng được phân tách rõ ràng hơn giữa các lớp, góp phần nâng cao hiệu quả phân loại.

Phương pháp này thường được áp dụng trong nhiều bài toán thực tế như nhận dạng khuôn mặt, nhận dạng giọng nói và phân loại văn bản. Do đó, LDA không chỉ đóng vai trò là một kỹ thuật giảm chiều mà còn là công cụ hỗ trợ hiệu quả cho các mô hình học máy có giám sát.

1.5.1 Cơ sở lý thuyết

Ta xét bài toán phân loại nhị phân (binary classification) với hai nhãn: $\{01, 02\}$. Đặt:

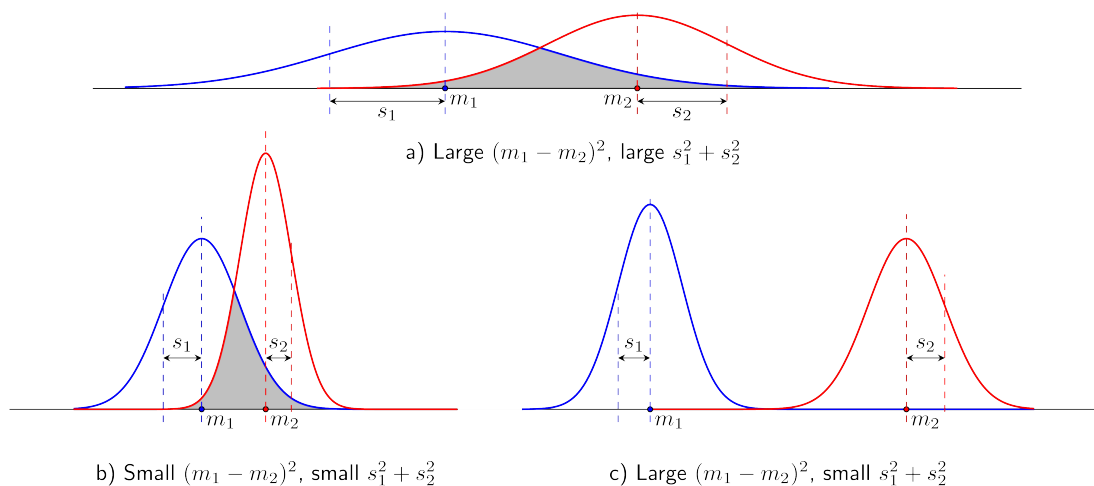
- X_1 : Tập dữ liệu có nhãn 01
- X_2 : Tập dữ liệu có nhãn 02

Giả sử khi chiếu dữ liệu xuống một chiều bất kỳ, ta có:

- Kỳ vọng (mean): m_1, m_2
- Phương sai (variance): s_1^2, s_2^2

Phân tích độ phân biệt trong các trường hợp của (m_1, m_2) và (s_1^2, s_2^2) như trong hình minh họa sau.

Ta xét ba trường hợp minh họa bằng đồ thị:



Hình 1.6: Minh họa các trường hợp phân biệt của LDA

- (A): $|m_1 - m_2|$ lớn nhưng s_1, s_2 cũng lớn $\rightarrow$ dữ liệu phân tán mạnh $\rightarrow$ vùng chồng lấn lớn $\rightarrow$ độ phân biệt thấp

Ta xét ba trường hợp minh họa bằng đồ thị:

- **(A):** $|m_1 - m_2|$ lớn nhưng s_1, s_2 cũng lớn $\rightarrow$ dữ liệu phân tán mạnh $\rightarrow$ vùng chồng lấn lớn $\rightarrow$ **độ phân biệt thấp**.
- **(B):** s_1, s_2 nhỏ nhưng $|m_1 - m_2|$ cũng nhỏ $\rightarrow$ dữ liệu gần nhau $\rightarrow$ vùng chồng lấn lớn $\rightarrow$ **độ phân biệt thấp**.
- **(C):** s_1, s_2 nhỏ và $|m_1 - m_2|$ lớn $\rightarrow$ vùng chồng lấn ít $\rightarrow$ **độ phân biệt cao**.

Từ các trường hợp trên, ta rút ra:

- **Phương sai trong lớp** (within-class variances): $N_j s_j^2$ với $N_j = |X_j|$ là số mẫu trong lớp X_j . Nếu nhỏ $\rightarrow$ dữ liệu tập trung $\rightarrow$ dễ phân biệt.
- **Phương sai giữa lớp** (between-class variance): $|m_1 - m_2|^2$. Nếu lớn $\rightarrow$ hai lớp phân bố xa nhau $\rightarrow$ dễ phân biệt.

Dữ liệu sẽ có khả năng phân biệt cao nếu:

- **Within-class variance nhỏ:** dữ liệu trong từng lớp ít phân tán.
- **Between-class variance lớn:** khoảng cách giữa các lớp lớn.

Các bước thực hiện

Để thực hiện phân tích Linear Discriminant Analysis (LDA), chúng ta làm theo các bước sau:

1. Bước 1: Tính S_W và S_B

Trong bước này, ta tính phương sai trong lớp (within-class variance) S_W và phương sai giữa các lớp (between-class variance) S_B .

- **Phương sai trong lớp (within-class variance):**

$$S_W = \sum_{x_n \in C_1} (x_n - m_1)(x_n - m_1)^T + \sum_{x_n \in C_2} (x_n - m_2)(x_n - m_2)^T$$

S_W thể hiện sự phân tán trong nội lớp, tức là sự biến động của các điểm trong lớp so với kỳ vọng của lớp đó.

- **Phương sai giữa các lớp (between-class variance):**

$$S_B = (m_1 - m_2)(m_1 - m_2)^T$$

S_B thể hiện sự phân tán giữa các lớp, tức là khoảng cách giữa các kỳ vọng so với trung tâm của hai lớp.

2. Bước 2: Tính $A = S_W^{-1}S_B$

$$A = S_W^{-1}S_B$$

Ma trận A này sẽ giúp ta tìm ra vector phân biệt tốt ưu trong không gian đặt biểu.

3. Bước 3: Tính $L = \max\{\lambda_i\}$, với $i = 1, 2, \dots, d$

$$L = \max\{\lambda_1, \lambda_2, \dots, \lambda_d\}$$

Chú ý: Với mỗi giá trị riêng λ , sẽ có một số vector riêng tương ứng.

4. Bước 4: Chọn w sao cho

Sau khi có giá trị riêng lớn nhất, ta chọn vector phân biệt tốt ưu sao cho:

$$(m_1 - m_2)^T w = L$$

Vector w được tính như sau:

$$w = S_W^{-1}(m_1 - m_2) \cdot \beta \quad (\text{với } \beta \geq 0)$$

Trong đó β là hệ số điều chỉnh, giúp tối ưu hóa sự phân biệt giữa các lớp.

Hàm mất mát

Hàm mất mát trong LDA có thể được định nghĩa dưới dạng tỷ lệ giữa phương sai giữa các lớp và phương sai trong lớp, đó:

$$J(w) = \frac{(m_1 - m_2)^2 w}{w^T S_W w}$$

Hàm mất mát này đo lường sự phân biệt giữa các lớp trong không gian giảm chiều được chiếu theo vector w .

Hàm tối ưu

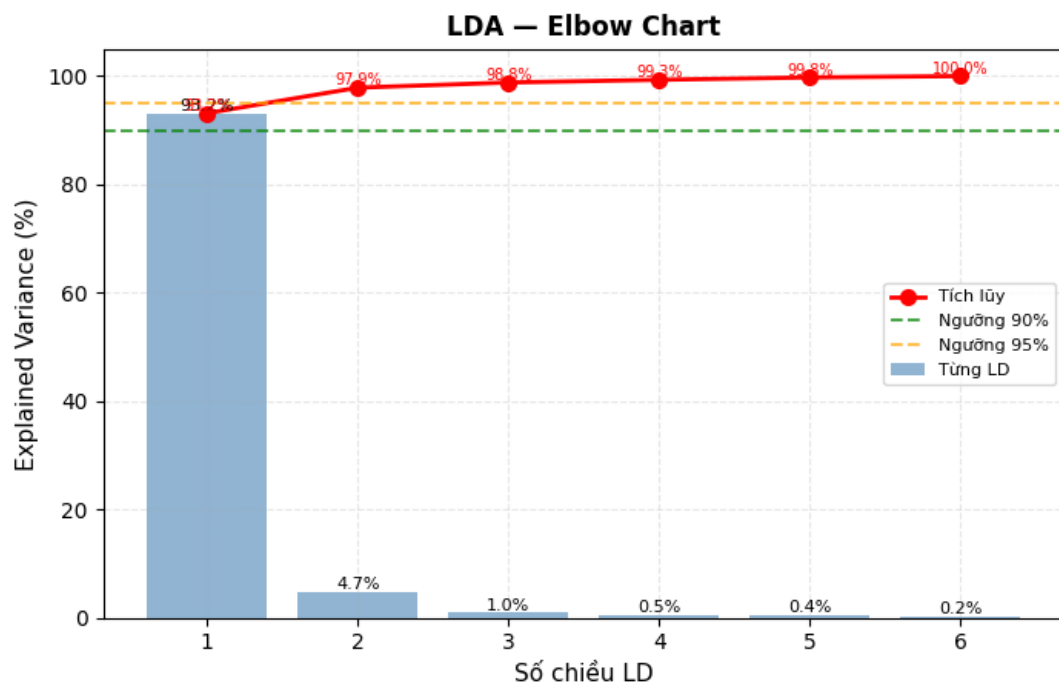
Để tối ưu hóa hàm mất mát $J(w)$, ta cần tìm giá trị của w sao cho hàm mất mát đạt cực đại. Bằng cách tính đạo hàm của $J(w)$ theo w và giải bài toán tối ưu, ta thu được:

$$w = S_W^{-1}(m_1 - m_2)$$

Bằng cách giải bài toán tối ưu này, ta tìm được vector phân biệt w sao cho độ phân biệt giữa các lớp là cao nhất, tức là phương sai giữa các lớp là lớn nhất trong khi phương sai trong lớp là nhỏ nhất.

1.5.2 Trục quan hóa dữ liệu và đánh giá kết quả

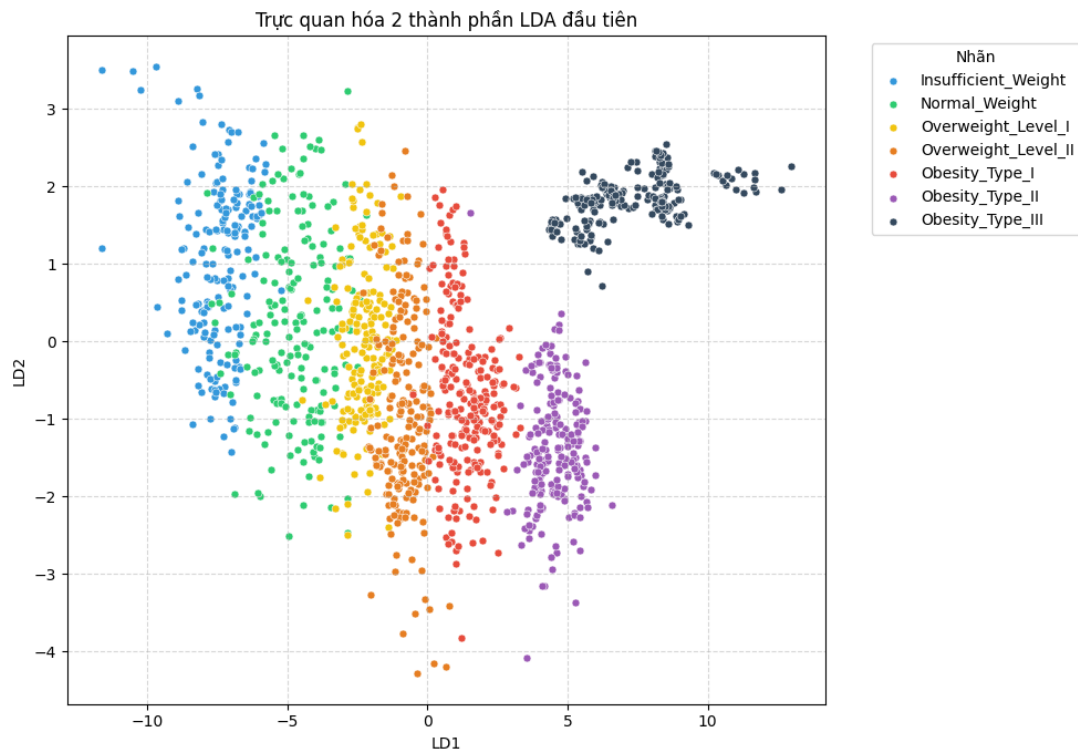
Cũng như ở phương pháp giảm chiều PCA, nhóm lựa chọn số lượng thành phần phân biệt tuyến tính sao cho tổng phương sai tích lũy đạt ít nhất 95%. Kết quả thu được như sau:



Hình 1.7: Biểu đồ phương sai tích lũy

Từ biểu đồ trên, sử dụng 2 thành phần phân biệt tuyến tính đầu tiên đã có thể giữ lại trên 95% thông tin của dữ liệu.

Áp dụng phương pháp LDA ta thu được kết quả sau:



Hình 1.8: Kết quả trực quan hóa sau khi áp dụng LDA

Kết quả cho thấy sau khi áp dụng LDA cho thấy dữ liệu được phân tách tương đối rõ ràng trong không gian hai chiều LD1-LD2. Trong đó, LD1 đóng vai trò chủ đạo trong việc phân tách giữa các lớp, Điều này cho thấy LDA đã cải thiện đáng kể khả năng phân biệt giữa các lớp so với không gian đặc trưng ban đầu, tuy nhiên vẫn còn hạn chế trong các vùng dữ liệu có phân bố gần nhau do bản chất tuyến tính của phương pháp.

1.6 So sánh PCA với LDA

Bảng 1.1: So sánh PCA và LDA trên bộ dữ liệu NObeyesdad

Tiêu chí	PCA	LDA
Sử dụng thông tin nhãn	Không	Có
Mục tiêu tối ưu	Phương sai dữ liệu	Phân biệt giữa các lớp
Khả năng tách lớp	Kém	Tốt hơn
Phù hợp bài toán phân loại	Không	Có

Quan sát trực quan cho thấy **LDA phù hợp hơn PCA** cho bộ dữ liệu này. Cụ thể, PCA chỉ tối ưu phương sai tổng thể mà không quan tâm đến nhãn lớp, dẫn đến các class bị **chồng lấn nghiêm trọng** trên không gian 2 chiều ($PC1 = 44.5\%$, $PC2 = 13.1\%$).

Trong khi đó, LDA tận dụng thông tin nhãn để tối đa hóa khoảng cách giữa các lớp, giúp các class **phân tách rõ ràng hơn** trên mặt phẳng $LD1-LD2$. Vậy nên với bộ dữ liệu này, sử dụng phương pháp LDA là tối ưu nhất

Chương 2

Các mô hình phân loại, phân cụm và chuyển về dạng hồi quy

2.1 Thuật toán phân cụm K-means

Một số ký hiệu toán học

Giả sử có N điểm dữ liệu là $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N] \in \mathbb{R}^{d \times N}$ và $K < N$ là số cluster chúng ta muốn phân chia. Chúng ta cần tìm các center $\mathbf{m}_1, \mathbf{m}_2, \dots, \mathbf{m}_K \in \mathbb{R}^{d \times 1}$ và label của mỗi điểm dữ liệu.

Với mỗi điểm dữ liệu $\mathbf{x}_i$, đặt $\mathbf{y}_i = [y_{i1}, y_{i2}, \dots, y_{iK}]$ là *label vector*, trong đó $y_{ik} = 1$ nếu $\mathbf{x}_i$ thuộc cụm k , và $y_{ij} = 0$ với mọi $j \neq k$. Như vậy, mỗi vector $\mathbf{y}_i$ có đúng một phần tử bằng 1, tương ứng với cụm mà $\mathbf{x}_i$ được gán vào.

Ràng buộc của $\mathbf{y}_i$ có thể viết dưới dạng toán học như sau:

$$y_{ik} \in \{0, 1\}, \quad \sum_{k=1}^K y_{ik} = 1 \quad (2.1)$$

2.1.1 Hàm mất mát và bài toán tối ưu

Nếu ta coi tâm (center) $\mathbf{m}_k$ là điểm đại diện của mỗi cluster và xấp xỉ tất cả các điểm được phân vào cluster này bằng $\mathbf{m}_k$, thì một điểm dữ liệu $\mathbf{x}_i$ được phân vào cluster k sẽ

có vector sai số là $(\mathbf{x}_i - \mathbf{m}_k)$. Chúng ta mong muốn sai số này có độ lớn nhỏ nhất, nên ta sẽ tìm cách để bình phương khoảng cách Euclid sau đây đạt giá trị nhỏ nhất:

$$\|\mathbf{x}_i - \mathbf{m}_k\|_2^2$$

Hơn nữa, vì $\mathbf{x}_i$ được phân vào cluster k (nghĩa là $y_{ik} = 1$ và $y_{ij} = 0$ với mọi $j \neq k$), biểu thức bình phương khoảng cách trên có thể được viết lại một cách tổng quát hơn như sau:

$$y_{ik}\|\mathbf{x}_i - \mathbf{m}_k\|_2^2 = \sum_{j=1}^K y_{ij}\|\mathbf{x}_i - \mathbf{m}_j\|_2^2$$

Từ đó, sai số (hàm mất mát – loss function) cho toàn bộ N điểm dữ liệu sẽ là:

$$\mathcal{L}(\mathbf{Y}, \mathbf{M}) = \sum_{i=1}^N \sum_{j=1}^K y_{ij}\|\mathbf{x}_i - \mathbf{m}_j\|_2^2$$

Trong đó $\mathbf{Y} = [\mathbf{y}_1; \mathbf{y}_2; \dots; \mathbf{y}_N]$, $\mathbf{M} = [\mathbf{m}_1, \mathbf{m}_2, \dots, \mathbf{m}_K]$ lần lượt là các ma trận được tạo bởi label vector của mỗi điểm dữ liệu và center của mỗi cluster. Hàm số mất mát trong bài toán K-means clustering của chúng ta là hàm $\mathcal{L}(\mathbf{Y}, \mathbf{M})$ với ràng buộc như được nêu trong phương trình (1).

Tóm lại, chúng ta cần tối ưu bài toán sau:

$$\mathbf{Y}, \mathbf{M} = \arg \min_{\mathbf{Y}, \mathbf{M}} \sum_{i=1}^N \sum_{j=1}^K y_{ij}\|\mathbf{x}_i - \mathbf{m}_j\|_2^2 \quad (2.2)$$

$$\text{subject to: } y_{ij} \in \{0, 1\} \quad \forall i, j; \quad \sum_{j=1}^K y_{ij} = 1 \quad \forall i$$

2.1.2 Thuật toán tối ưu hàm mất mát

Bài toán (2) là một bài toán tối ưu phức tạp do có các ràng buộc, đặc biệt là biến rời rạc. Đây là một bài toán thuộc loại **mixed-integer programming (MIP)**, vốn rất khó để tìm nghiệm tối ưu toàn cục.

Tuy nhiên, trong thực tế, ta thường chỉ cần nghiệm gần đúng hoặc cực tiểu cục bộ là đủ. Một cách tiếp cận đơn giản và hiệu quả là *lấp xen kẽ*: lần lượt cố định một biến và tối ưu biến còn lại. Cụ thể:

- Giải bài toán tối ưu theo biến $\mathbf{Y}$, khi đã cố định giá trị của $\mathbf{M}$.

- Sau đó, giải bài toán tối ưu theo biến $\mathbf{M}$, khi đã cố định giá trị của $\mathbf{Y}$.

Quá trình này được lặp lại cho đến khi hội tụ. Đây là một chiến lược phổ biến và thực tiễn trong các bài toán tối ưu có nhiều biến phụ thuộc lẫn nhau.

Cố định $\mathbf{M}$, tìm $\mathbf{Y}$

Giả sử đã tìm được các centers, hãy tìm các label vector để hàm mất mát đạt giá trị nhỏ nhất. Điều này tương đương với việc tìm cluster cho mỗi điểm dữ liệu.

Khi các centers là cố định, bài toán tìm label vector cho toàn bộ dữ liệu có thể được chia nhỏ thành bài toán tìm label vector cho từng điểm dữ liệu $\mathbf{x}_i$ như sau:

$$\mathbf{y}_i = \arg \min_{\mathbf{y}_i} \sum_{j=1}^K y_{ij} \|\mathbf{x}_i - \mathbf{m}_j\|_2^2 \quad (2.3)$$

$$\text{subject to: } y_{ij} \in \{0, 1\} \quad \forall j; \quad \sum_{j=1}^K y_{ij} = 1$$

Vì chỉ có một phần tử của label vector $\mathbf{y}_i$ bằng 1 nên bài toán (3) có thể tiếp tục được viết dưới dạng đơn giản hơn:

$$j = \arg \min_j \|\mathbf{x}_i - \mathbf{m}_j\|_2^2$$

Vì $\|\mathbf{x}_i - \mathbf{m}_j\|_2^2$ chính là bình phương khoảng cách tính từ điểm $\mathbf{x}_i$ tới center $\mathbf{m}_j$, ta có thể kết luận rằng mỗi điểm $\mathbf{x}_i$ thuộc vào cluster có center gần nó nhất! Từ đó ta có thể dễ dàng suy ra label vector của từng điểm dữ liệu.

Cố định $\mathbf{Y}$, tìm $\mathbf{M}$

Giả sử đã tìm được cluster cho từng điểm, hãy tìm center mới cho mỗi cluster để hàm mất mát đạt giá trị nhỏ nhất.

Một khi chúng ta đã xác định được label vector cho từng điểm dữ liệu, bài toán tìm center cho mỗi cluster được rút gọn thành:

$$\mathbf{m}_j = \arg \min_{\mathbf{m}_j} \sum_{i=1}^N y_{ij} \|\mathbf{x}_i - \mathbf{m}_j\|_2^2 \quad (2.4)$$

Tối đây, ta có thể tìm nghiệm bằng phương pháp giải đạo hàm bằng 0, vì hàm cần tối ưu là một hàm liên tục và có đạo hàm xác định tại mọi điểm. Và quan trọng hơn, hàm này là hàm convex (lồi) theo $\mathbf{m}_j$ nên chúng ta sẽ tìm được giá trị nhỏ nhất và điểm tối ưu tương ứng.

Đặt $l(\mathbf{m}_j)$ là hàm bên trong dấu arg min, ta có đạo hàm:

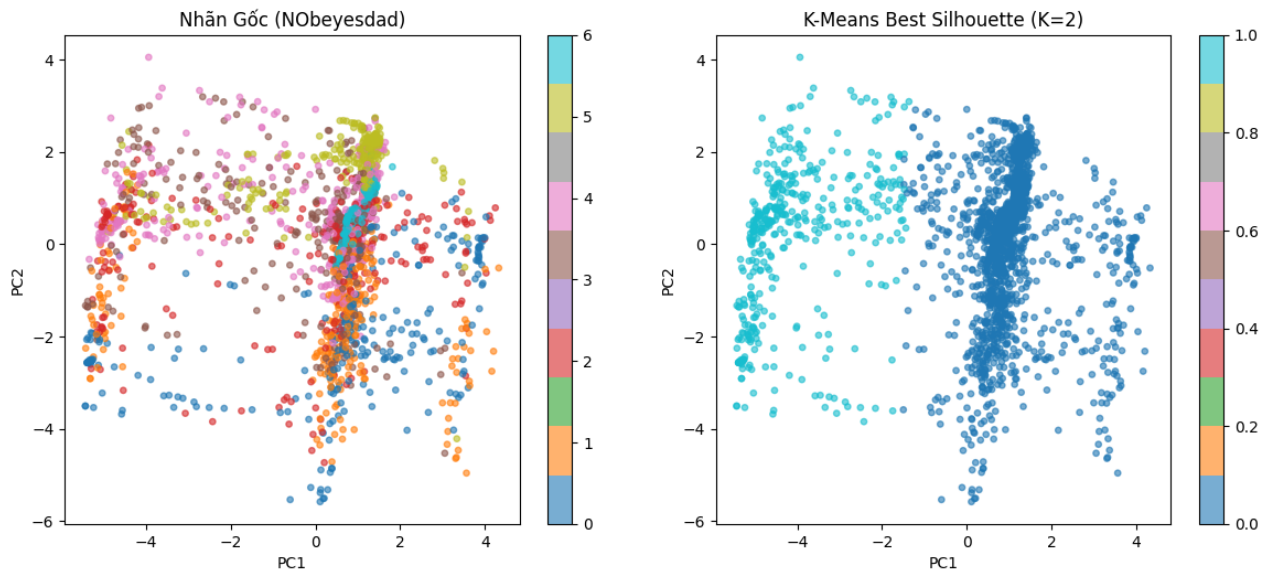
$$\frac{\partial l(\mathbf{m}_j)}{\partial \mathbf{m}_j} = 2 \sum_{i=1}^N y_{ij}(\mathbf{m}_j - \mathbf{x}_i)$$

Giải phương trình đạo hàm bằng 0 ta có:

$$\mathbf{m}_j \sum_{i=1}^N y_{ij} = \sum_{i=1}^N y_{ij} \mathbf{x}_i$$

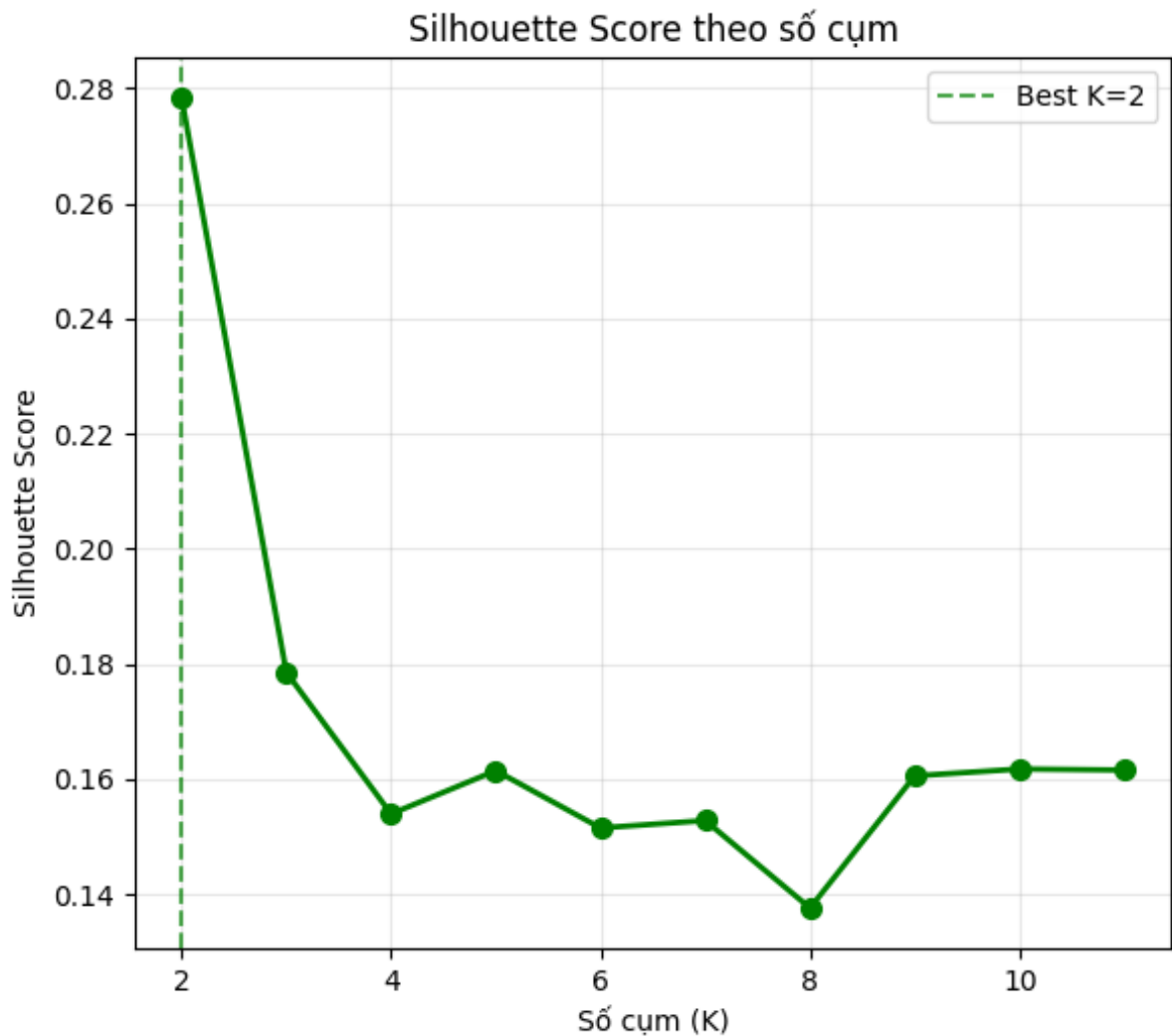
$$\Rightarrow \mathbf{m}_j = \frac{\sum_{i=1}^N y_{ij} \mathbf{x}_i}{\sum_{i=1}^N y_{ij}}$$

2.1.3 Trực quan hóa và đánh giá kết quả



Hình 2.1: trực quan hóa phân cụm với dữ liệu PCA

Đánh giá Silhouette Score



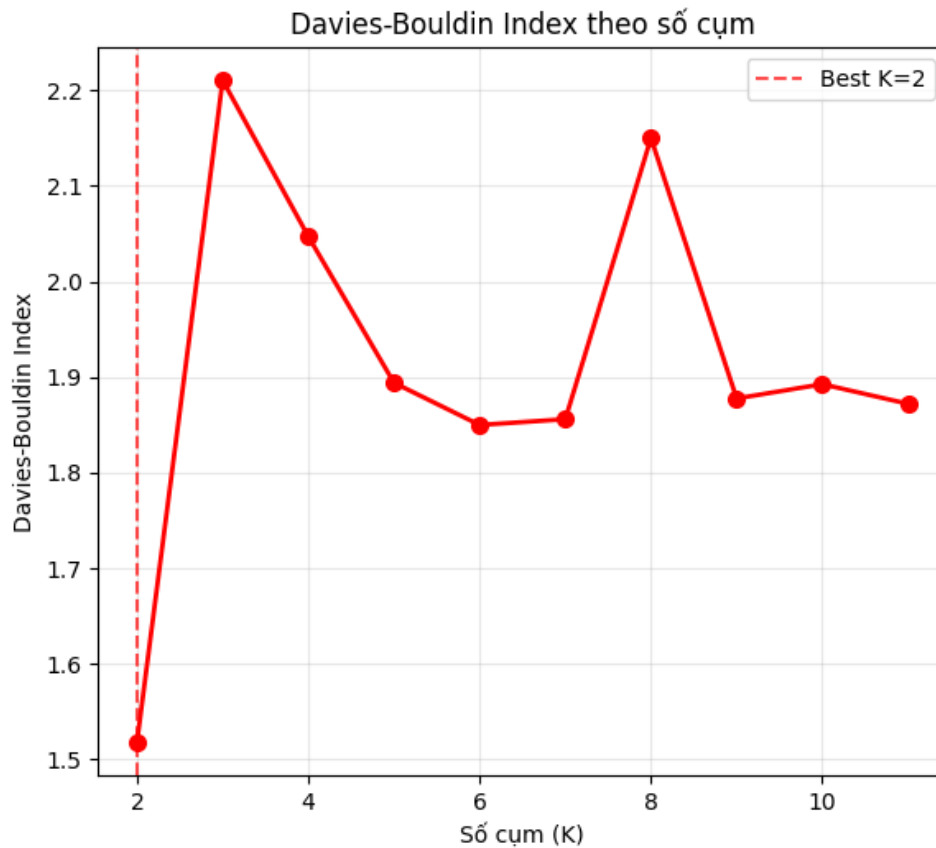
Hình 2.2: Biểu đồ Silhouette score theo số lượng cụm K

Từ biểu đồ, có thể quan sát:

- Tại $K = 2$, Silhouette Score đạt giá trị cao nhất ($\approx 0,28$), cho thấy sự tách biệt rõ ràng nhất giữa các cụm.
- Khi K tăng từ 2 đến 4, chỉ số giảm mạnh, phản ánh sự phân tán của dữ liệu khi chia thành nhiều cụm hơn.
- Từ $K = 4$ trở đi, Silhouette Score dao động trong khoảng $[0,14, 0,16]$ và không cho thấy xu hướng cải thiện rõ rệt.
- Mức giảm mạnh nhất xảy ra tại $K = 8$ ($\approx 0,14$), trước khi phục hồi nhẹ.

Nhận xét: $K = 2$ là lựa chọn tối ưu theo tiêu chí Silhouette Score.

Đánh giá Davies-Bouldin Index



Hình 2.3: Biểu đồ Davies-Bouldin index theo số lượng cụm K

Từ biểu đồ, có thể quan sát:

- Tại $K = 2$, DBI đạt giá trị thấp nhất ($\approx 1,52$), khẳng định phân cụm tốt nhất.
- DBI tăng vọt tại $K = 3$ ($\approx 2,21$), cho thấy sự chồng lấp tăng đáng kể.
- Từ $K = 4$ đến $K = 7$, DBI có xu hướng giảm dần về mức $\approx 1,85$, tạo thành một vùng ổn định tương đối.
- Tại $K = 8$, DBI tăng trở lại ($\approx 2,15$), sau đó giảm và ổn định quanh 1,88 với $K \geq 9$.

Nhận xét: $K = 2$ là lựa chọn tối ưu theo tiêu chí Davies-Bouldin Index.

Kết luận

Dựa trên phân tích đồng thời cả hai chỉ số đánh giá, kết quả được tổng hợp trong Bảng 2.1:

Cả hai chỉ số đều nhất quán chỉ ra rằng **$K = 2$ là số cụm tối ưu** cho tập dữ liệu này:

Bảng 2.1: Tổng hợp chỉ số phân cụm tại các giá trị K tiêu biểu

K	Silhouette Score	Davies-Bouldin Index	Đánh giá tổng thể
2	0,28	1,52	Tốt nhất
3	0,18	2,21	Kém
4	0,15	2,04	Trung bình
6	0,15	1,85	Trung bình
8	0,14	2,15	Kém
10	0,16	1,88	Trung bình

1. **Silhouette Score** đạt cực đại tại $K = 2$ ($\approx 0,28$), nghĩa là các điểm dữ liệu nằm đúng cụm của chúng và tách biệt tốt với cụm lân cận.
2. **Davies-Bouldin Index** đạt cực tiểu tại $K = 2$ ($\approx 1,52$), nghĩa là các cụm gọn và ít chồng lấp nhất.
3. Với $K \geq 3$, không có giá trị nào vượt trội rõ ràng so với $K = 2$ theo cả hai tiêu chí đồng thời.

Do đó, mô hình phân cụm với $K = 2$ được khuyến nghị làm cấu hình cuối cùng trong bộ thực nghiệm này. Tuy nhiên, cần nhấn mạnh rằng đây là lựa chọn tốt nhất tương đối, tức là tốt nhất trong số các K đã thử, chứ chưa phải một mô hình phân cụm chất lượng cao về mặt tuyệt đối. Cụ thể:

- Silhouette Score $\approx 0,28$ ở mức *trung bình thấp* (ngưỡng tốt cần $> 0,50$).
- Davies-Bouldin Index $\approx 1,52$ ở mức *trung bình* (ngưỡng tốt cần $< 1,00$).

kết quả này thiếu ý nghĩa thực tiễn vì không phản ánh được sự đa dạng của 7 mức độ béo phì trong dữ liệu. Điều này cho thấy K-Means không phải lựa chọn phù hợp cho bài toán này, và cần xem xét các phương pháp phân cụm khác có khả năng xử lý tốt hơn với dữ liệu có cấu trúc phức tạp và nhiều lớp gần nhau như Gaussian Mixture Model hoặc DBSCAN.

Một số mô hình phân loại

2.2 K-Nearest neighbors (KNN)

2.2.1 Giới thiệu

K-Nearest Neighbors (KNN) là một trong những thuật toán học có giám sát (*Supervised Learning*) đơn giản và phổ biến nhất trong lĩnh vực Machine Learning. Thuật toán này không xây dựng mô hình tường minh trong quá trình huấn luyện mà lưu trữ toàn bộ dữ liệu huấn luyện để sử dụng trực tiếp trong giai đoạn dự đoán. Mọi quá trình tính toán của thuật toán chủ yếu được thực hiện khi cần dự đoán nhãn cho dữ liệu mới. KNN có thể được áp dụng cho cả hai dạng bài toán học có giám sát là phân loại (*Classification*) và hồi quy (*Regression*).

2.2.2 Cách thuật toán hoạt động

Cách hoạt động của thuật toán KNN trong bài toán phân loại nhị phân và phân loại đa lớp về cơ bản là tương tự nhau. Để dễ hình dung, ta xét bài toán phân loại nhị phân với hai nhãn là 0 và 1. Các bước thực hiện của thuật toán được mô tả như sau:

1. Với mỗi điểm dữ liệu $\mathbf{x}$ chưa biết nhãn trong tập Validation hoặc Test, thuật toán sẽ tìm các điểm lân cận gần nhất trong tập huấn luyện.
2. Tính khoảng cách từ điểm dữ liệu $\mathbf{x}$ đến từng điểm $\mathbf{x}_i$ trong tập huấn luyện, từ đó tạo thành một mảng chứa các khoảng cách tương ứng.
3. Sắp xếp các khoảng cách theo thứ tự tăng dần và lưu lại chỉ số của các điểm dữ liệu tương ứng.
4. Chọn ra K điểm có khoảng cách nhỏ nhất với $\mathbf{x}$, tức là K láng giềng gần nhất của điểm cần dự đoán.

5. Nhân dự đoán y_{pred} được xác định theo quy tắc:

$$y_{\text{pred}} = \begin{cases} 1, & \text{nếu } \frac{1}{K} \sum_{i=1}^K y_{(i)} \geq 0.5 \\ 0, & \text{nếu } \frac{1}{K} \sum_{i=1}^K y_{(i)} < 0.5 \end{cases}$$

trong đó $y_{(i)}$ là nhãn của láng giềng gần thứ i .

2.2.3 Bài toán phân loại đa lớp

Cho tập dữ liệu huấn luyện:

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$$

trong đó:

$$\mathbf{x}_i \in \mathbb{R}^d, \quad y_i \in \{1, 2, \dots, C\}$$

với:

- N là số lượng mẫu dữ liệu huấn luyện,
- d là số chiều của không gian đặc trưng,
- C là số lớp cần phân loại.

Mục tiêu của bài toán là xây dựng mô hình có khả năng dự đoán nhãn $\hat{y}$ cho một điểm dữ liệu mới $\mathbf{x}$:

$$\hat{y} \in \{1, 2, \dots, C\}$$

Trong thuật toán K-Nearest Neighbors (KNN), việc dự đoán nhãn được thực hiện dựa trên các điểm dữ liệu gần nhất trong không gian đặc trưng. Ý tưởng cơ bản của thuật toán cho rằng các điểm dữ liệu nằm gần nhau thường có xu hướng thuộc cùng một lớp.

2.2.4 Hàm tính khoảng cách

Để xác định mức độ tương đồng giữa các điểm dữ liệu, thuật toán KNN sử dụng các độ đo khoảng cách. Khoảng cách được sử dụng phổ biến nhất là khoảng cách Euclidean (chuẩn L_2):

$$d_i = \|\mathbf{x} - \mathbf{x}_i\|_2 = \sqrt{\sum_{j=1}^d (x_j - x_{i,j})^2}$$

Trong đó:

- $\mathbf{x}$ là điểm dữ liệu cần dự đoán,
- $\mathbf{x}_i$ là điểm dữ liệu trong tập huấn luyện,
- d_i là khoảng cách giữa $\mathbf{x}$ và $\mathbf{x}_i$.

Ngoài khoảng cách Euclidean, thuật toán KNN còn có thể sử dụng các độ đo khoảng cách khác tùy thuộc vào đặc điểm của dữ liệu:

- **Khoảng cách Manhattan** (chuẩn L_1):

$$d_i = \sum_{j=1}^d |x_j - x_{i,j}|$$

- **Khoảng cách ư tổng quát bậc p :**

$$d_i = \left(\sum_{j=1}^d |x_j - x_{i,j}|^p \right)^{1/p}$$

Việc lựa chọn hàm khoảng cách phù hợp có ảnh hưởng đáng kể đến hiệu quả của mô hình KNN.

2.2.5 Tìm K láng giềng gần nhất và dự đoán nhãn

Sau khi tính khoảng cách từ điểm dữ liệu $\mathbf{x}$ đến toàn bộ các điểm trong tập huấn luyện, ta thu được tập khoảng cách:

$$D = (d_1, d_2, \dots, d_N)$$

Các khoảng cách này được sắp xếp theo thứ tự tăng dần để lựa chọn ra K láng giềng gần nhất:

$$\mathcal{N}_K(\mathbf{x}) = \{(\mathbf{x}_{(1)}, y_{(1)}), (\mathbf{x}_{(2)}, y_{(2)}), \dots, (\mathbf{x}_{(K)}, y_{(K)})\}$$

Với mỗi lớp $c \in \{1, 2, \dots, C\}$, số phiếu bầu được xác định bởi:

$$v_c = \sum_{i=1}^K \mathbf{1}[y_{(i)} = c]$$

trong đó:

$$\mathbf{1}[y_{(i)} = c] = \begin{cases} 1, & \text{nếu } y_{(i)} = c \\ 0, & \text{ngược lại} \end{cases}$$

Nhãn dự đoán của điểm dữ liệu $\mathbf{x}$ được xác định theo quy tắc đa số:

$$\hat{y} = \arg \max_{c \in \{1, \dots, C\}} v_c$$

Ngoài ra, xác suất hậu nghiệm của lớp c cũng có thể được ước lượng thông qua tỉ lệ phiếu bầu:

$$\hat{P}(y = c | \mathbf{x}) = \frac{v_c}{K} = \frac{1}{K} \sum_{i=1}^K \mathbf{1}[y_{(i)} = c]$$

2.2.6 KNN có trọng số

Trong KNN cơ bản, tất cả các láng giềng đều có mức độ ảnh hưởng như nhau. Tuy nhiên, trong nhiều trường hợp, các điểm dữ liệu nằm gần hơn thường có mức độ ảnh hưởng lớn hơn đến kết quả dự đoán.

Để cải thiện hiệu quả mô hình, thuật toán Weighted KNN gán cho mỗi láng giềng một trọng số tỉ lệ nghịch với khoảng cách:

$$w_i = \frac{1}{d_i + \varepsilon}, \quad \varepsilon > 0$$

trong đó ε là một hằng số nhỏ nhằm tránh trường hợp chia cho 0.

Khi đó, số phiếu bầu có trọng số của lớp c được xác định bởi:

$$v_c^{(w)} = \sum_{i=1}^K w_i \cdot \mathbf{1}[y_{(i)} = c]$$

Nhãn dự đoán cuối cùng được xác định theo công thức:

$$\hat{y} = \arg \max_{c \in \{1, \dots, C\}} v_c^{(w)}$$

Phương pháp này giúp giảm ảnh hưởng của các điểm dữ liệu ở xa và thường cho kết quả phân loại tốt hơn so với KNN thông thường.

2.2.7 Lựa chọn tham số K

Tham số K đóng vai trò quan trọng và ảnh hưởng trực tiếp đến hiệu suất của mô hình KNN. Trong thực tế, giá trị K thường được lựa chọn thông qua phương pháp Cross Validation hoặc theo kinh nghiệm:

$$K \approx \sqrt{N}$$

Ảnh hưởng của tham số K được thể hiện như sau:

Trường hợp	Đặc điểm
K quá nhỏ	Mô hình nhạy với nhiễu và dễ xảy ra hiện tượng overfitting
K quá lớn	Mô hình trở nên quá tổng quát và có thể dẫn đến hiện tượng underfitting

Trong nhiều bài toán thực tế, giá trị K thường được chọn là số lẻ nhằm giảm khả năng xảy ra hòa phiếu trong quá trình phân loại.

2.3 Softmax Regression

2.3.1 Mô hình hóa xác suất

Khác với hồi quy Logistic chỉ áp dụng cho bài toán phân loại nhị phân, hồi quy Softmax (Softmax Regression) có khả năng xử lý bài toán phân loại đa lớp. Với mỗi mẫu dữ liệu đầu vào $\mathbf{x}$, mô hình tính xác suất để mẫu thuộc từng lớp thông qua hàm Softmax.

Giả sử bài toán có K lớp ($K \geq 2$), xác suất để mẫu $\mathbf{x}$ thuộc lớp k được xác định như sau:

$$p(y = k | \mathbf{x}; \mathbf{W}, \mathbf{b}) = \frac{e^{\mathbf{w}_k^T \mathbf{x} + b_k}}{\sum_{j=1}^K e^{\mathbf{w}_j^T \mathbf{x} + b_j}}$$

Trong đó:

- $\mathbf{W}$: ma trận trọng số có kích thước $d \times K$, với d là số đặc trưng của dữ liệu đầu vào.

- $\mathbf{w}_k$: vector trọng số tương ứng với lớp k .
- $\mathbf{b}$: vector hệ số điều chỉnh (bias).
- K : số lượng lớp cần phân loại.

2.3.2 Hàm Softmax

Hàm Softmax có nhiệm vụ chuyển đổi các giá trị đầu ra của mô hình tuyến tính thành phân phối xác suất trên các lớp:

$$\text{Softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

với:

$$\mathbf{z} = \mathbf{W}^T \mathbf{x} + \mathbf{b}$$

Trong đó, z_k biểu diễn điểm số (logit) của lớp k . Sau khi áp dụng hàm Softmax, tổng xác suất của tất cả các lớp luôn bằng 1:

$$\sum_{k=1}^K p(y = k|\mathbf{x}) = 1$$

2.3.3 Quyết định lớp dựa trên xác suất

Trong hồi quy Softmax, nhãn thực tế thường được biểu diễn dưới dạng *one-hot encoding*, nghĩa là vector chỉ chứa đúng một phần tử bằng 1 và các phần tử còn lại bằng 0.

Sau khi tính được xác suất cho từng lớp, mô hình sẽ lựa chọn lớp có xác suất lớn nhất làm kết quả dự đoán:

$$\hat{y} = \arg \max_k p(y = k|\mathbf{x})$$

hay tương đương:

$$\hat{y} = \arg \max_k a_k$$

với:

$$\mathbf{a} = \text{softmax}(\mathbf{W}^T \mathbf{x} + \mathbf{b})$$

Trong đó $\mathbf{a}$ là vector xác suất dự đoán của mô hình.

2.3.4 Hàm mất mát (Loss Function)

Hàm mất mát phổ biến trong hồi quy Softmax là *Cross-Entropy Loss* (hàm entropy chéo), được sử dụng để đo mức độ sai khác giữa phân phối xác suất dự đoán và nhãn thực tế.

Hàm mất mát được xác định như sau:

$$L(\mathbf{W}, \mathbf{b}) = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_{ik} \log(p(y = k | \mathbf{x}_i; \mathbf{W}, \mathbf{b}))$$

Trong đó:

- N : số lượng mẫu trong tập huấn luyện.
- y_{ik} : bằng 1 nếu mẫu thứ i thuộc lớp k , ngược lại bằng 0.
- $p(y = k | \mathbf{x}_i)$: xác suất dự đoán mẫu thứ i thuộc lớp k .

Mục tiêu của quá trình huấn luyện là tối thiểu hóa giá trị hàm mất mát này để cải thiện khả năng phân loại của mô hình.

2.3.5 Tối ưu hóa mô hình

Quá trình tối ưu hóa tham số trong hồi quy Softmax thường sử dụng các thuật toán như Gradient Descent, Stochastic Gradient Descent (SGD), Adam hoặc các biến thể khác.

Gradient của hàm mất mát đối với ma trận trọng số được tính như sau:

$$\frac{\partial L}{\partial \mathbf{W}} = \frac{1}{N} \mathbf{X}^T (\mathbf{A} - \mathbf{Y})$$

Trong đó:

- $\mathbf{X}$: ma trận dữ liệu đầu vào.
- $\mathbf{A}$: ma trận xác suất dự đoán.
- $\mathbf{Y}$: ma trận biểu diễn nhãn thực tế dưới dạng one-hot encoding.

Nếu sử dụng thuật toán SGD, trọng số được cập nhật tại mỗi bước theo công thức:

$$\mathbf{W} = \mathbf{W} - \eta \mathbf{x}_i (\mathbf{a}_i - \mathbf{y}_i)^T$$

Trong đó:

- η : tốc độ học (*learning rate*).
- $\mathbf{x}_i$: vector đặc trưng của mẫu thứ i .
- $\mathbf{a}_i$: vector xác suất dự đoán của mẫu thứ i .
- $\mathbf{y}_i$: nhãn thực tế dưới dạng one-hot encoding.

2.4 Support vector machine (SVM)

2.4.1 Giới thiệu

Support Vector Machine (SVM) là một thuật toán máy học có giám sát (supervised learning) được sử dụng phổ biến trong các bài toán phân loại và hồi quy. Mục tiêu chính của SVM là tìm ra một siêu phẳng (hyperplane) tối ưu để phân tách các lớp dữ liệu sao cho khoảng cách giữa siêu phẳng và các điểm dữ liệu gần nhất của mỗi lớp (gọi là support vectors) là lớn nhất. Do đó, SVM thường mang lại hiệu quả cao trong các bài toán có dữ liệu phức tạp hoặc nhiều chiều.

2.4.2 Một số khái niệm cơ bản

- **Siêu phẳng (Hyperplane):**

Là một không gian con có chiều thấp hơn không gian đặc trưng, được biểu diễn bởi phương trình:

$$\mathbf{w}^\top \mathbf{x} + b = 0$$

Siêu phẳng này có vai trò phân chia không gian dữ liệu thành hai nửa tương ứng với hai lớp.

- **Biên (Margin):**

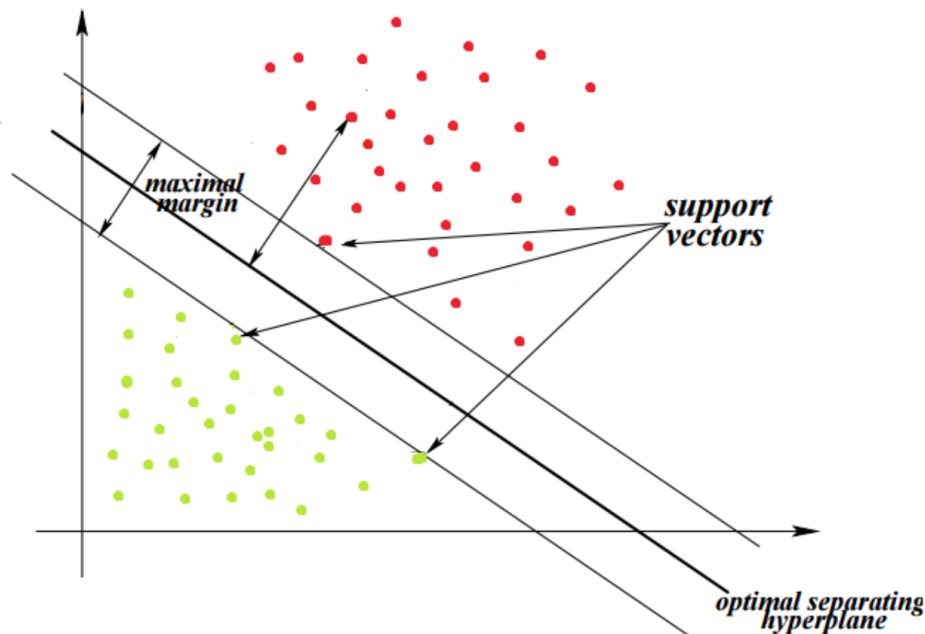
Là khoảng cách từ siêu phẳng đến điểm dữ liệu gần nhất thuộc mỗi lớp, được tính qua công thức:

$$\frac{|\mathbf{w}^\top \mathbf{x}_0 + b|}{\|\mathbf{w}\|_2}$$

trong đó $\|\mathbf{w}\|_2 = \sqrt{\sum_{i=1}^d w_i^2}$ với d là số chiều của không gian.

- **Các véc tơ hỗ trợ (Support vectors):**

Một điểm trong không gian véc tơ có thể được coi là một véc tơ từ gốc tọa độ tới điểm đó. Các điểm dữ liệu nằm trên hoặc gần nhất với siêu phẳng được gọi là véc tơ hỗ trợ, chúng ảnh hưởng đến vị trí và hướng của siêu phẳng. Các véc tơ này được sử dụng để tối ưu hóa lề và nếu xóa các điểm này, vị trí của siêu phẳng sẽ thay đổi. Một điểm lưu ý nữa là các véc tơ hỗ trợ phải chạm đều siêu phẳng.



Hình 2.4: Mô phỏng các véc tơ hỗ trợ

2.4.3 Bài toán tối ưu của SVM

Bài toán tối ưu trong Support Vector Machine (SVM) thực chất là bài toán tìm một siêu phẳng phân chia dữ liệu sao cho khoảng cách giữa hai lớp là lớn nhất, hay nói cách khác là tối đa hóa margin

Hard-Soft margin

Hard Margin là một phiên bản tối ưu trong điều kiện lý tưởng, tức bộ dữ liệu:

- Không có nhiễu.
- Dữ liệu phân tách tuyến tính hoàn hảo.

Trong thực tế, dữ liệu huấn luyện thường không tuyến tính phân biệt hoàn toàn, khiến việc áp dụng Hard-Margin không khả thi. Vì vậy, **Soft Margin** là một phiên

bản linh hoạt hơn của Hard Margin, cho phép sai số xảy ra bằng cách thêm biến bù ξ_i để đo lường mức độ vi phạm ràng buộc của từng điểm dữ liệu.

Hàm mục tiêu:

$$\min_{w, b, \xi} \quad \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \xi_i$$

Trong đó:

- $\frac{1}{2} \|w\|^2$: tối thiểu hoá để tối đa hoá khoảng cách phân tách (margin).
- $\sum_{i=1}^N \xi_i$: tổng mức độ vi phạm ràng buộc (lỗi phân loại).
- $C > 0$: tham số điều chỉnh sự đánh đổi giữa hai mục tiêu trên.

Ràng buộc mô hình:

$$y_i (w^\top x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad \forall i = 1, \dots, N$$

Để đưa hàm mục tiêu trên về dạng đơn giản hơn, ta áp dụng phương pháp nhân tử Lagrange. Hàm Lagrangian tương ứng là:

$$\mathcal{L}(w, b, \xi, \alpha, \beta) = \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \xi_i - \sum_{i=1}^N \alpha_i (y_i (w^\top x_i + b) - 1 + \xi_i) - \sum_{i=1}^N \beta_i \xi_i$$

với điều kiện: $\alpha_i \geq 0, \quad \beta_i \geq 0$.

Rút gọn và đưa về dạng đối ngẫu, ta có bài toán tối ưu:

$$\max_{\alpha} \quad \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j \langle x_i, x_j \rangle$$

$$\text{subject to:} \quad 0 \leq \alpha_i \leq C, \quad \sum_{i=1}^N \alpha_i y_i = 0$$

Kết quả tối ưu:

- Trọng số tối ưu:

$$w^* = \sum_{i=1}^N \alpha_i y_i x_i$$

- Bias tối ưu (tính trên các support vector thoả $0 < \alpha_k < C$):

$$b^* = y_k - w^{*\top} x_k$$

Chuyển các mô hình phân loại sang hồi quy

2.5 Chuyển đổi từ SVM sang SVM hồi quy (SVR)

2.5.1 Support Vector Regression (SVR)

Support Vector Regression (SVR) là phần mở rộng của thuật toán Support Vector Machine (SVM) để giải bài toán hồi quy. Khác với SVM phân loại nhằm tìm siêu phẳng tối ưu để phân tách hai lớp, SVR tìm một hàm hồi quy sao cho phần lớn điểm dữ liệu nằm trong một khoảng sai số ε cho phép.

Hàm hồi quy tuyến tính có dạng:

$$f(x) = w^T x + \gamma \quad (6.1)$$

Trong đó, w là vector trọng số, γ là hệ số điều chỉnh. Mục tiêu là làm cho giá trị dự đoán $f(x)$ gần với giá trị thực y trong một khoảng sai số ε :

$$|y - f(x)| \leq \varepsilon \quad (6.2)$$

Khi có các điểm nằm ngoài khoảng ε , SVR sử dụng các biến trượt ξ_i, ξ_i^* để đo độ sai lệch:

$$w \cdot x_i - y_i \leq \varepsilon + \xi_i \quad (6.3)$$

$$w^T x_i + \gamma - y_i \leq \varepsilon + \xi_i^* \quad (6.4)$$

$$\xi_i, \xi_i^* \geq 0 \quad (6.5)$$

Hàm mục tiêu tối ưu hóa của SVR là:

$$\min_{w, \gamma, \xi, \xi^*} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n (\xi_i + \xi_i^*) \quad (6.6)$$

Trong đó:

- C là tham số điều chỉnh, điều chỉnh mức đánh đổi giữa độ phẳng của mô hình và độ chính xác của dự đoán.
- $\|w\|^2$ là chuẩn L_2 giúp mô hình không quá phức tạp.
- $(\xi_i + \xi_i^*)$ là tổng độ vi phạm.

Hàm mất mát ε -insensitive được định nghĩa như sau:

$$|\xi|_\varepsilon = \begin{cases} 0 & \text{nếu } |\xi| \leq \varepsilon \\ |\xi| - \varepsilon & \text{nếu } |\xi| > \varepsilon \end{cases} \quad (6.7)$$

2.5.2 Chuyển đổi từ SVM phân loại sang SVR hồi quy

Mặc dù SVM chủ yếu được sử dụng trong bài toán phân loại, nguyên lý tối ưu hóa biên (margin) và sử dụng các điểm hỗ trợ (support vectors) có thể được mở rộng cho bài toán hồi quy như sau:

- **Mục tiêu:** SVM cố gắng tìm siêu phẳng phân cách với margin lớn nhất giữa các lớp. SVR tìm một hàm hồi quy với margin ε đủ rộng để bao phủ đa số dữ liệu.
- **Dạng bài toán:** SVM là phân loại rời rạc ($y \in \{-1, +1\}$), còn SVR là hồi quy liên tục ($y \in \mathbb{R}$).
- **Hàm mất mát:** SVM dùng hinge loss; SVR dùng epsilon-insensitive loss.
- **Tham số:** Cả hai đều sử dụng C để điều chỉnh mức độ phạt sai số, nhưng SVR còn thêm tham số ε kiểm soát biên độ không bị phạt.

2.5.3 Cách áp dụng mô hình SVR từ mô hình SVM phân loại

Trong phần này, nhóm thực hiện một phương pháp kết hợp giữa mô hình phân loại SVM và mô hình hồi quy SVR như sau:

1. Huấn luyện mô hình SVM phân loại với kernel RBF trên tập huấn luyện để phân biệt giữa các lớp dữ liệu.

2. Trích xuất giá trị `decision_function` từ SVM – đại diện cho khoảng cách có hướng tới siêu phẳng phân tách.
3. Sử dụng các giá trị `decision_function` làm mục tiêu hồi quy trong SVR.
4. Huấn luyện mô hình SVR với cùng tập dữ liệu gốc và giá trị đầu ra hồi quy là các giá trị này.
5. Dự đoán và đánh giá hiệu năng hồi quy bằng các chỉ số MSE và R^2 .

2.5.4 Các khái niệm cốt lõi trong SVR

SVR dựa trên một số nguyên lý chính để học các hàm phi tuyến hiệu quả:

- **Hàm kernel:** Dùng để ánh xạ dữ liệu vào không gian đặc trưng cao hơn. Thường dùng các kernel như: tuyến tính, RBF (Gaussian), polynomial hoặc sigmoid.
- **Tham số ε và C :**
 - ε xác định vùng “an toàn” không bị phạt sai số.
 - C điều chỉnh giữa độ chính xác và sự đơn giản của mô hình.
- **Support Vectors:** Là những điểm nằm đúng trên biên ε hoặc nằm ngoài nó – có ảnh hưởng trực tiếp đến hàm hồi quy.
- **Tối ưu hóa:** SVR giải bài toán tối ưu lồi có ràng buộc để tìm ra hàm hồi quy phù hợp với nhiều dữ liệu nhất trong khoảng ε .

2.5.5 Ý nghĩa của phương pháp

Việc chuyển đổi từ mô hình SVM sang SVR mang lại một số lợi ích. Cụ thể, SVR kế thừa cơ chế tối ưu hóa của SVM nhưng thay vì phân tách các lớp rời rạc, nó xấp xỉ một **hàm số liên tục** phản ánh mức độ béo phì theo thứ tự tự nhiên giữa các lớp. Hơn nữa, đầu ra liên tục cho phép áp dụng các công cụ đánh giá hồi quy (MSE, R^2), qua đó đo lường chính xác hơn mức độ sai lệch của dự đoán so với thực tế, điều mà các chỉ số phân loại thuần túy không thể hiện được.

2.6 Chuyển đổi từ Softmax phân loại sang Softmax hồi quy

2.6.1 Giới thiệu phương pháp Softmax hồi quy

Trong phần này, nhóm trình bày phương pháp chuyển đổi mô hình Softmax Classification sang dạng hồi quy nhằm khai thác thông tin xác suất liên tục, thay vì chỉ sử dụng nhãn dự đoán rời rạc.

Mô hình được sử dụng là **Multinomial Logistic Regression** (hay còn gọi là Softmax Regression). Thay vì chỉ trả về nhãn lớp dự đoán, mô hình cung cấp phân phối xác suất trên toàn bộ các lớp thông qua hàm `predict_proba()`, tức là với mỗi mẫu đầu vào x , mô hình trả về vector xác suất:

$$[P(y = 1 | x), P(y = 2 | x), \dots, P(y = K | x)]$$

trong đó K là số lớp phân loại và $\sum_{k=1}^K P(y = k | x) = 1$.

2.6.2 Chuyển đổi từ Softmax Classification sang hồi quy

Để chuyển bài toán phân loại sang bài toán hồi quy, nhóm áp dụng **giá trị kỳ vọng** (Expected Value) của phân phối xác suất đầu ra. Cụ thể, nếu ký hiệu:

- $P(y = k | x)$: xác suất mẫu x thuộc lớp k ,
- $k \in \{1, 2, \dots, K\}$: chỉ số thứ tự của lớp,

thì giá trị hồi quy được tính theo công thức:

$$\hat{y} = \sum_{k=1}^K k \cdot P(y = k | x)$$

Kết quả $\hat{y}$ là một **giá trị thực liên tục**, phản ánh mức độ dự đoán của mô hình thay vì một nhãn phân loại rời rạc.

Phương pháp này đặc biệt phù hợp với bài toán phân loại béo phì vì các lớp mang tính **thứ tự** (ordinal): từ mức thiếu cân (*Insufficient Weight*) đến các mức béo phì nghiêm trọng (*Obesity Type III*). Việc tính kỳ vọng cho phép mô hình biểu diễn sự chuyển tiếp liên tục giữa các mức độ này, thay vì xử lý chúng như các lớp độc lập hoàn toàn.

2.6.3 Cách áp dụng hồi quy từ mô hình Softmax phân loại

Nhóm thực hiện phương pháp chuyển đổi từ bài toán phân loại sang hồi quy theo các bước sau:

1. Huấn luyện mô hình *Multinomial Logistic Regression* trên tập dữ liệu huấn luyện, nhằm học phân phối xác suất của các lớp thông qua hàm Softmax.
2. Sử dụng hàm `predict_proba()` để thu được véc-tơ xác suất dự đoán $\hat{p}_k$ cho từng lớp k ứng với mỗi mẫu dữ liệu đầu vào.
3. Tính giá trị hồi quy liên tục bằng kỳ vọng của phân phối xác suất:

$$\hat{y} = \sum_{k=1}^K k \cdot \hat{p}_k \quad (2.5)$$

trong đó K là số lớp, k là nhãn lớp thứ k , và $\hat{p}_k$ là xác suất dự đoán tương ứng.

4. Sử dụng các giá trị kỳ vọng $\hat{y}$ thu được làm nhãn hồi quy liên tục để đánh giá khả năng biểu diễn thứ tự của mô hình.
5. Dự đoán và đánh giá hiệu năng hồi quy bằng các chỉ số MSE và R^2 .

2.6.4 Ý nghĩa của phương pháp

Phương pháp này mang lại hai lợi ích chính. Thứ nhất, nó cho phép khai thác **toàn bộ thông tin xác suất** mà mô hình Softmax Classification cung cấp, thay vì chỉ lấy nhãn của lớp có xác suất cao nhất. Thứ hai, việc tạo ra đầu ra liên tục giúp áp dụng được các công cụ đánh giá hồi quy (MSE, R^2), qua đó đánh giá khả năng mô hình biểu diễn mối quan hệ có thứ tự giữa các mức độ béo phì một cách toàn diện hơn so với đánh giá phân loại thuần túy.

Chương 3

Nhận xét và đánh giá kết quả thực nghiệm

3.1 Đánh giá mô hình phân loại

Nhóm tiến hành đánh giá mô hình thông qua các chỉ số gồm Accuracy, F1-score và ma trận nhầm lẫn (Confusion Matrix). Đây là những độ đo phổ biến trong các bài toán phân loại nhằm đánh giá mức độ chính xác và khả năng phân biệt giữa các lớp của mô hình.

- **Độ chính xác (Accuracy)**

Accuracy là tỷ lệ số mẫu được dự đoán đúng trên tổng số mẫu của tập dữ liệu, dùng để đánh giá độ chính xác tổng quát của mô hình.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Trong đó:

TP: dự đoán đúng lớp dương.

TN: dự đoán đúng lớp âm.

FP: dự đoán sai từ âm sang dương.

FN: dự đoán sai từ dương sang âm.

- **Chỉ số F1-score**

F1-score là trung bình điều hòa giữa Precision và Recall, thường được sử dụng khi dữ liệu mất cân bằng.

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

Trong đó:

$$Precision = \frac{TP}{TP + FP}$$

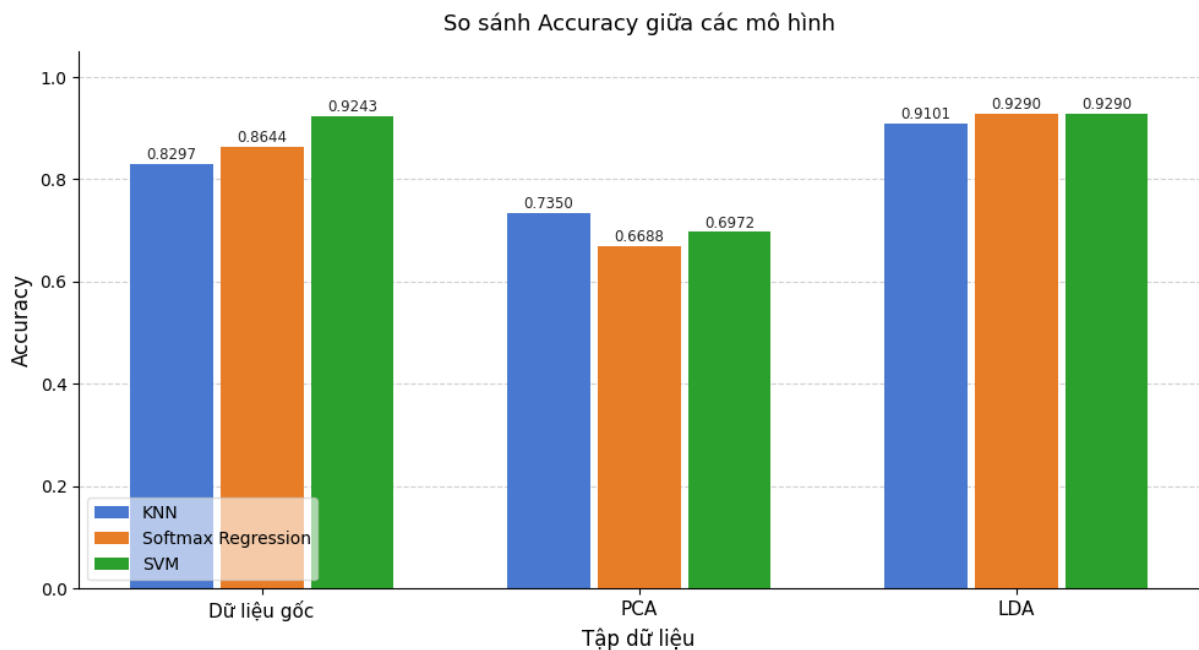
$$Recall = \frac{TP}{TP + FN}$$

Precision: tỷ lệ dự đoán đúng trong các mẫu được dự đoán dương.

Recall: khả năng phát hiện đúng các mẫu dương.

F1-score: cân bằng giữa Precision và Recall.

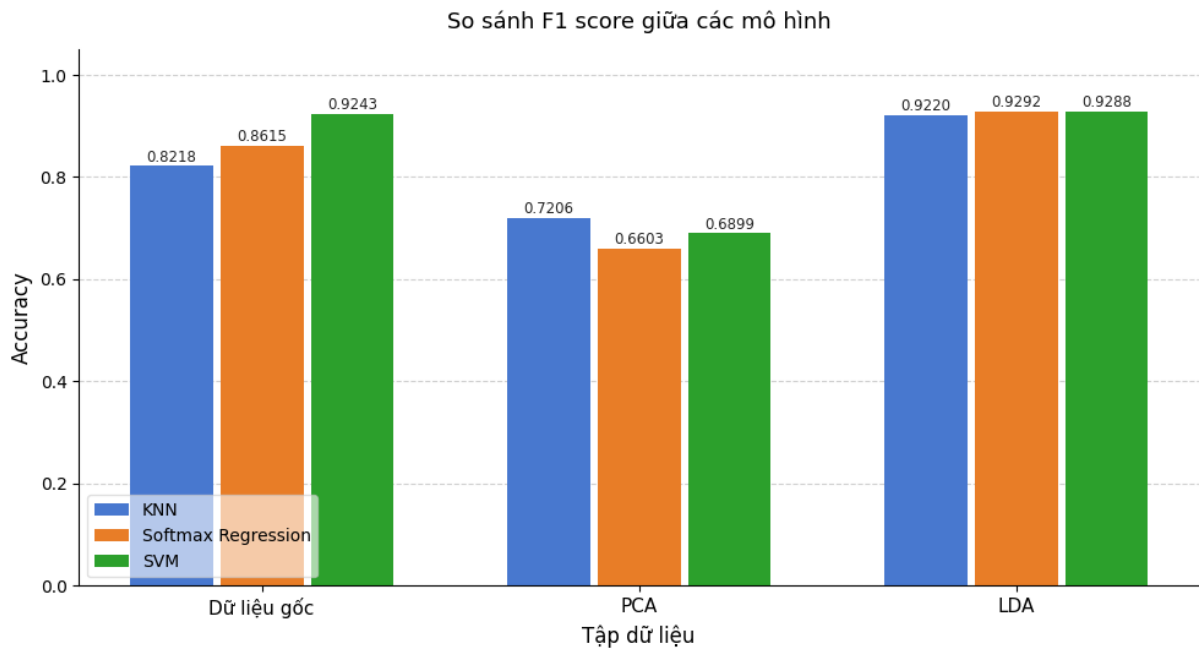
Kết quả độ chính xác các mô hình



Hình 3.1: So sánh độ chính xác các mô hình

• Tổng quan hiệu suất theo tập dữ liệu

- LDA cho kết quả tốt nhất ở cả 3 mô hình, với accuracy dao động từ 0.9209 – 0.9274, cho thấy LDA giúp các mô hình phân loại hiệu quả hơn đáng kể.
- Dữ liệu gốc cho kết quả trung bình, accuracy khoảng 0.8596 – 0.8660.



Hình 3.2: So sánh F1 score các mô hình

- PCA cho kết quả kém nhất, đặc biệt Softmax Regression chỉ đạt 0.6688, cho thấy PCA có thể làm mất thông tin quan trọng với một số mô hình.

• So sánh giữa các mô hình

- Softmax Regression đạt hiệu suất cao nhất trên LDA với accuracy 0.9274 và F1 score 0.9276, đồng thời cho kết quả cạnh tranh trên dữ liệu gốc (0.8644).
- KNN cho kết quả ổn định trên dữ liệu gốc và LDA, đặc biệt trên LDA đạt accuracy 0.9243, nhưng bị ảnh hưởng đáng kể khi dùng PCA (0.7981).
- SVM có hiệu suất tốt nhất trên dữ liệu gốc (0.8660), ổn định trên LDA (0.9209), nhưng giảm xuống 0.7535 khi dùng PCA.
- Softmax Regression là mô hình nhạy cảm nhất với cách biểu diễn dữ liệu — hoạt động tốt trên dữ liệu gốc và LDA, nhưng giảm mạnh nhất xuống 0.6688 khi dùng PCA.

Kết quả chỉ số F1 các mô hình

• Tổng quan theo tập dữ liệu

- LDA tiếp tục cho F1-score cao nhất ở cả 3 mô hình, dao động 0.9211 – 0.9276, khẳng định LDA là phương pháp biểu diễn dữ liệu tốt nhất cho bài toán này.

- Dữ liệu gốc cho kết quả trung bình, F1-score khoảng 0.8550 – 0.8659.
- PCA vẫn cho kết quả thấp nhất, đặc biệt Softmax Regression chỉ đạt 0.6603, cho thấy PCA làm giảm khả năng phân loại đáng kể.

• So sánh giữa các mô hình

- Softmax Regression đạt F1-score cao nhất trên LDA (0.9276) và dữ liệu gốc (0.8615), nhưng bị ảnh hưởng nặng nhất bởi PCA khi giảm mạnh xuống 0.6603.
- KNN cho kết quả ổn định nhất trên cả 3 tập dữ liệu, đặc biệt dẫn đầu trên PCA với F1-score 0.7896.
- SVM có F1-score tốt trên dữ liệu gốc (0.8659) và ổn định trên LDA (0.9211), cho thấy mô hình này cân bằng tốt giữa Precision và Recall.

Kết quả Recall và Precision

Bảng 3.1: So sánh Recall và Precision của các mô hình phân loại

Phương pháp	KNN		Softmax Regression		SVM	
	Precision	Recall	Precision	Recall	Precision	Recall
Dữ liệu gốc	0.8550	0.8574	0.8612	0.8631	0.8681	0.8642
PCA	0.7954	0.7973	0.6536	0.6643	0.7372	0.7552
LDA	0.9379	0.9365	0.9262	0.9259	0.9189	0.9193

- **Softmax + LDA** đạt hiệu năng tốt nhất trong toàn bảng với Precision = 0.9262 và Recall = 0.9259, đồng thời cũng được xác nhận qua Accuracy = 0.9274 và F1 = 0.9276 cao nhất — cho thấy LDA tạo ra không gian đặc trưng giúp Softmax phân tách các lớp hiệu quả nhất.
- **KNN + LDA** và **SVM + LDA** cũng đạt kết quả rất tốt và tương đương nhau (KNN: Precision = 0.9379, Recall = 0.9365; SVM: Precision = 0.9189, Recall = 0.9193), cho thấy LDA mang lại lợi ích đồng đều cho cả ba mô hình.
- **SVM** có hiệu năng ổn định nhất trên dữ liệu gốc (Precision = 0.8681, Recall = 0.8642), nhỉnh hơn KNN và Softmax một chút, phản ánh khả năng tìm siêu phẳng phân tách tối ưu của SVM ngay cả khi chưa giảm chiều.

- **Softmax Regression** bị ảnh hưởng nặng nề nhất bởi PCA, với Precision chỉ đạt 0.6536 và Recall = 0.6643 — thấp nhất toàn bảng — phản ánh hạn chế của mô hình tuyến tính khi đặc trưng bị nén không phù hợp.

Đánh giá K-fold Cross-Validation

K-fold Cross-Validation là phương pháp đánh giá mô hình phổ biến trong machine learning nhằm kiểm tra khả năng tổng quát hóa của mô hình trên dữ liệu chưa thấy trước. Phương pháp này hoạt động bằng cách chia tập dữ liệu thành K phần bằng nhau (gọi là folds), trong đó mỗi lần huấn luyện sẽ sử dụng K-1 folds làm tập train và 1 fold còn lại làm tập validation để đánh giá mô hình. Quá trình này được lặp lại K lần sao cho mỗi fold đều được sử dụng một lần làm validation, sau đó kết quả đánh giá cuối cùng được tính bằng trung bình của tất cả các lần chạy. K-fold Cross-Validation giúp giảm sự phụ thuộc vào một cách chia dữ liệu duy nhất, từ đó cho kết quả đánh giá ổn định và đáng tin cậy hơn. Trong quá trình thực hiện.

Bảng 3.2: Kết quả đánh giá K-Fold Cross-Validation ($k = 5$) trên ba tập dữ liệu

Dataset	Mô hình	Accuracy		F1-score		Gap	Trạng thái
		CV Train	CV Val	CV Val	Test		
Gốc	Softmax	0.8885	0.8646	0.8603	0.8579	0.0239	Ổn định
	SVM	0.9584	0.9411	0.9390	0.9512	0.0173	Ổn định
	KNN	0.8546	0.8199	0.8092	0.8197	0.0347	Ổn định
PCA	Softmax	0.6794	0.6533	0.6324	0.6548	0.0261	Ổn định
	SVM	0.7244	0.6838	0.6650	0.6840	0.0406	Ổn định
	KNN	0.7818	0.7359	0.7164	0.7180	0.0459	Ổn định
LDA	Softmax	0.9379	0.9357	0.9329	0.9233	0.0022	Ổn định
	SVM	0.9418	0.9309	0.9275	0.9264	0.0108	Ổn định
	KNN	0.9298	0.9167	0.9112	0.9068	0.0130	Ổn định

- **bộ dữ liệu gốc:** Trên tập dữ liệu gốc, cả ba mô hình đều cho kết quả tương đối ổn định với Gap đều nhỏ hơn ngưỡng 0.05. Trong đó, mô hình SVM đạt hiệu suất tốt nhất với CV Val Accuracy = 0.9411 và Test Accuracy = 0.9527, đồng thời có Gap nhỏ (0.0173), cho thấy khả năng tổng quát hóa tốt. Softmax cũng duy trì được sự ổn định với Gap = 0.0239, trong khi KNN cho kết quả thấp hơn nhưng vẫn không xuất hiện dấu hiệu overfitting đáng kể.

- **bộ dữ liệu PCA:** Sau khi giảm chiều bằng PCA, hiệu suất của cả ba mô hình đều suy giảm so với tập dữ liệu gốc. Softmax, SVM và KNN lần lượt đạt CV Val Accuracy là 0.6533, 0.6838 và 0.7359. Mặc dù các mô hình vẫn duy trì trạng thái ổn định với Gap đều nhỏ hơn 0.05, PCA dường như làm mất đi một phần thông tin quan trọng phục vụ phân loại, dẫn đến khả năng dự đoán giảm đáng kể.
- **bộ dữ liệu LDA:** LDA mang lại kết quả tốt và ổn định trên cả ba mô hình. Softmax đạt CV Val Accuracy = 0.9357, SVM đạt 0.9309, và KNN đạt 0.9167, với Gap đều rất nhỏ (dao động từ 0.0022 đến 0.0130), phản ánh khả năng tổng quát hóa tốt. So với PCA, LDA cho hiệu suất vượt trội hơn đáng kể, cho thấy phương pháp giảm chiều có giám sát này bảo toàn tốt hơn thông tin phân biệt giữa các lớp trong bài toán phân loại.

Kết luận

- **Phương pháp giảm chiều LDA** mang lại hiệu quả tốt nhất cho cả ba mô hình phân loại. Các mô hình huấn luyện trên dữ liệu LDA đều đạt Accuracy và F1-score cao hơn đáng kể so với dữ liệu PCA, đồng thời duy trì khả năng tổng quát hóa tốt. Điều này cho thấy LDA — một phương pháp giảm chiều có giám sát — tận dụng hiệu quả thông tin nhãn lớp để tạo ra không gian đặc trưng có tính phân tách cao hơn.
- **Hiệu suất trên dữ liệu LDA:** Cả ba mô hình đều đạt kết quả tốt khi huấn luyện trên dữ liệu LDA. Softmax Regression đạt Accuracy = 0.9290 và F1-score = 0.9292; SVM đạt Accuracy = 0.9290 và F1-score = 0.9288; trong khi KNN đạt Accuracy = 0.9101 và F1-score = 0.9220. Sự chênh lệch rất nhỏ giữa Accuracy và F1-score ở cả ba mô hình phản ánh khả năng phân loại đồng đều và ổn định trên các lớp dữ liệu.
- **Hiệu suất trên dữ liệu gốc:** Trên tập dữ liệu gốc chưa qua giảm chiều, SVM vẫn là mô hình mạnh nhất với Accuracy = 0.9243 và F1-score = 0.9243. Softmax Regression đạt Accuracy = 0.8644 và F1-score = 0.8615, trong khi KNN có kết quả thấp hơn với Accuracy = 0.8297 và F1-score = 0.8218.
- **Phương pháp PCA** cho kết quả thấp nhất trong ba biến thể dữ liệu. Accuracy và F1-score của cả ba mô hình đều suy giảm đáng kể: KNN đạt Accuracy = 0.7350 và F1-score = 0.7206; SVM đạt Accuracy = 0.6972 và F1-score = 0.6899; Softmax

Regression đạt kết quả thấp nhất với $\text{Accuracy} = 0.6688$ và $\text{F1-score} = 0.6603$. PCA, do không sử dụng thông tin nhãn trong quá trình giảm chiều, dường như làm mất đi một phần thông tin phân biệt lớp quan trọng.

- **Mô hình tốt nhất tổng thể:** Xét trên cả hai tiêu chí Accuracy và F1-score, tổ hợp Softmax Regression + LDA và SVM + LDA đạt hiệu suất cao nhất với $\text{Accuracy} = 0.9290$ và F1-score lần lượt là 0.9292 và 0.9288, vượt nhẹ so với SVM trên dữ liệu gốc ($\text{Accuracy} = 0.9243$, $\text{F1-score} = 0.9243$). Trong đó, Softmax Regression + LDA nổi bật là lựa chọn tối ưu nhờ đạt F1-score cao nhất (0.9292), thể hiện sự cân bằng tốt giữa độ chính xác và khả năng tổng quát hóa trong bài toán phân loại đa lớp về mức độ béo phì.

3.2 Đánh giá mô hình hồi quy

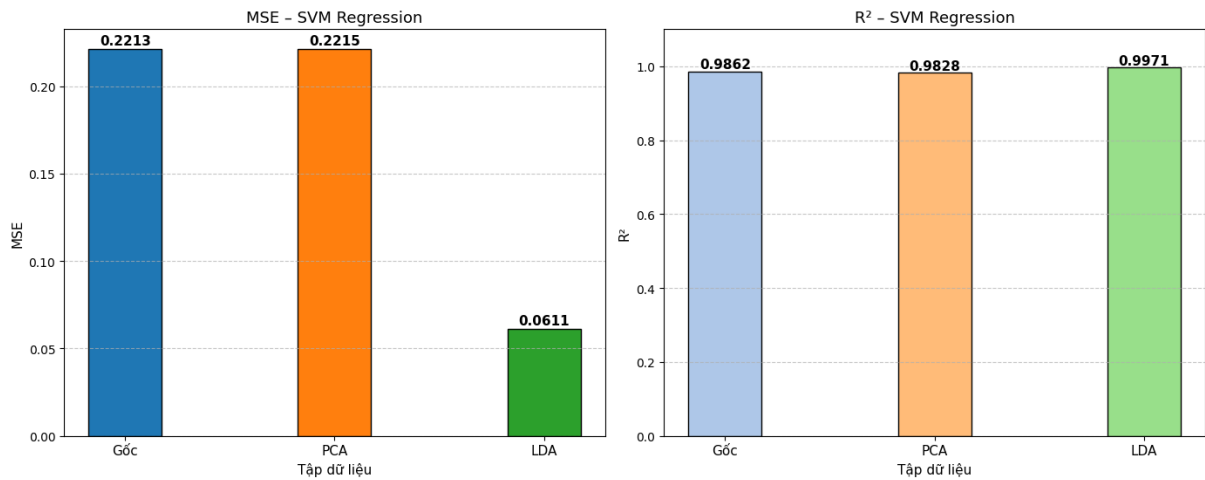
Sau khi chuyển sang dạng hồi quy, nhóm đánh giá hiệu quả mô hình thông qua hai chỉ số định lượng:

- **Mean Squared Error (MSE):** đo lường sai số bình phương trung bình giữa nhãn thực tế và giá trị dự đoán liên tục:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

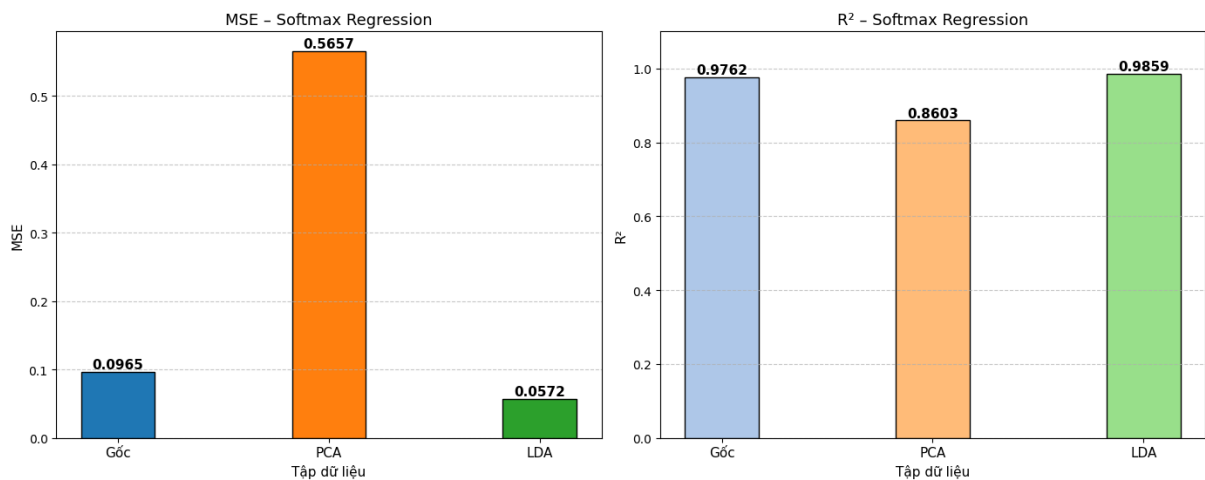
- **Hệ số xác định R^2 :** đánh giá mức độ mô hình giải thích được phương sai của dữ liệu, với $R^2 \in [0, 1]$, giá trị càng gần 1 cho thấy mô hình càng phù hợp với dữ liệu.

Kết quả thực nghiệm SVM hồi quy (SVR)



Hình 3.3: Biểu đồ cột cho MSE và R^2 SVM

Kết quả thực nghiệm Softmax hồi quy



Hình 3.4: Biểu đồ cột cho MSE và R^2 Softmax hồi quy

Softmax hồi quy

Kết quả thực nghiệm cho thấy mô hình Softmax Regression đạt hiệu năng hồi quy tốt trên tập dữ liệu gốc và LDA, nhưng kém hơn đáng kể trên tập PCA. Cụ thể, tập dữ liệu gốc có $MSE = 0,0965$ và $R^2 = 0,9762$, cho thấy mô hình xấp xỉ tốt các giá trị thực. Tập LDA cho kết quả tốt nhất với MSE thấp nhất $= 0,0572$ và R^2 cao nhất $= 0,9859$, cho thấy việc giảm chiều theo hướng phân biệt lớp giúp mô hình hồi quy hoạt động hiệu quả hơn.

Ngược lại, tập PCA cho kết quả kém nhất với $MSE = 0,5657$ và $R^2 = 0,8603$, cho thấy việc giảm chiều theo phương sai tối đa không bảo toàn tốt thông tin phân loại có thứ tự cần thiết cho bài toán này.

SVM hồi quy (SVR)

Mô hình SVR thể hiện sự ổn định cao hơn giữa các tập dữ liệu so với Softmax Regression. Tập dữ liệu gốc và PCA cho MSE gần tương đương nhau, lần lượt là 0,2213 và 0,2215, với R^2 đạt 0,9862 và 0,9828, cho thấy mô hình không bị ảnh hưởng nhiều bởi việc giảm chiều PCA.

Tập LDA tiếp tục cho kết quả vượt trội nhất với $MSE = 0,0611$ và $R^2 = 0,9971$, gần như xấp xỉ hoàn hảo giá trị thực. Điều này khẳng định rằng LDA là phương pháp giảm chiều phù hợp hơn cho cả hai mô hình trong bài toán phân loại béo phì có tính thứ tự.

Kết luận

- Đối với mô hình SVM hồi quy (SVR), phương pháp giảm chiều LDA cho kết quả tốt nhất với MSE thấp nhất đạt 0.0572 và R^2 cao nhất đạt 0.9859. Trong khi đó, PCA cho kết quả kém nhất với MSE lên tới 0.5657 và R^2 chỉ đạt 0.8603, cho thấy PCA làm giảm đáng kể khả năng hồi quy của SVR.
- Đối với mô hình Softmax hồi quy, LDA cũng cho kết quả vượt trội với MSE thấp nhất đạt 0.0611 và R^2 cao nhất đạt 0.9971. Dữ liệu gốc và PCA cho kết quả tương đương nhau với MSE lần lượt là 0.2213 và 0.2215, R^2 lần lượt là 0.9862 và 0.9828, cho thấy PCA hầu như không cải thiện hiệu suất so với dữ liệu gốc đối với mô hình này.
- Phương pháp giảm chiều LDA một lần nữa khẳng định hiệu quả vượt trội so với PCA và dữ liệu gốc trên cả hai mô hình hồi quy. LDA giúp giảm mạnh sai số MSE và nâng cao đáng kể hệ số R^2 , chứng tỏ khả năng giữ lại thông tin hồi quy quan trọng tốt hơn so với PCA.
- Mô hình tốt nhất: Tổ hợp Softmax hồi quy + LDA được xác định là mô hình tốt nhất trong thí nghiệm hồi quy, với MSE đạt 0.0611 và R^2 đạt 0.9971, cho thấy mô hình này có khả năng dự đoán chính xác và giải thích tốt phương sai của dữ liệu.

3.3 Một số hạn chế và phương hướng phát triển

Trong quá trình thực hiện dự án, nhóm thấy rằng có một số hạn chế như sau.

- Bộ dữ liệu được sử dụng gồm hơn 2000 đối tượng nghiên cứu với 7 nhãn đầu ra khác nhau, do đó bài toán phân loại có độ phức tạp tương đối cao.
- Quá trình lựa chọn và tối ưu các siêu tham số cho mô hình, chẳng hạn như số láng giềng k trong KNN hay hàm Kernel của SVM, vẫn chưa đạt được mức tối ưu hoàn toàn.

Trong thời gian tới, dự án này có thể phát triển theo nhiều hướng.

- Thử nghiệm thêm các mô hình nâng cao như Random Forest, XGBoost... Nhằm so sánh và cải thiện hiệu suất so với các mô hình hiện tại như SVM, KNN và Softmax Regression.
- Tối ưu hóa siêu tham số bằng các phương pháp như Grid Search, Random Search hoặc Bayesian Optimization để tìm ra bộ tham số phù hợp nhất cho từng mô hình. Đặc biệt đối với KNN (giá trị k), SVM (Kernel, C , γ) và Softmax Regression (learning rate, số lớp ẩn, dropout).

Tài liệu tham khảo

- [1] TS. Cao Văn Chung, Giáo trình môn học Học máy. Trường đại học Khoa học tự nhiên. Đại học Quốc gia Hà Nội.